Contents

1 Contest **1**

How big can n be? 1 • Which technique? 1 • Template 2 • Terminal setup 2 • Reading input 2

2 C++ quick reference **2**

Sorting 2 • Useful algorithms 2 • Containers 2 • Strings and characters 2 • Bits 2 • Lambdas 3 • Ordered set 3 • Hash for pair keys 3

3 Simulation and parsing **3**

Grids 3 • Time and dates 3 • Splitting text 3 • Number bases 3

4 Sorting and greedy **4**

Greedy moves 4 • Finding the sort order 4 • Most non-overlapping intervals 4 • Merging intervals 4 • Most intervals at the same time 4 • Jobs with deadlines 4 • Matching with the smallest that fits 4 • The answer is one of the input values 5

5 Searching and arrays **5**

Binary search on the answer 5 • Binary search on real numbers 5 • Finding a peak 5 • Searching in sorted data 5 • Prefix sums 5 • Difference array 5 • Two pointers / sliding window 5 • Coordinate compression 5 • Monotonic stack 5 • Largest rectangle in a histogram 6 • Sliding window minimum 6

6 Dynamic programming **6**

How to build a DP 6 • Memoized recursion 6 • Best subarray sum 6 • 0/1 knapsack and subset sum 6 • Coin change 6 • Longest increasing subsequence 7 • Longest common subsequence 7 • Edit distance 7 • Paths in a grid 7 • Cutting a string into words 7 • Cutting an array into k parts 7 • Range sums inside a DP 7 • Weighted intervals 7 • Interval DP 8 • Bitmask DP 8 • Digit DP 8

7 Graphs **8**

Which shortest path? 8 • Reading a graph 8 • BFS 8 • Grid BFS 9 • 0-1 BFS 9 • Dijkstra 9 • Bellman–Ford 9 • Floyd–Warshall 9 • DFS and connected components 9 • Topological sort 9 • Union-Find (DSU) 10 • Minimum spanning tree (Kruskal) 10

8 Trees and more graphs **10**

Tree facts 10 • Rooting a tree 10 • DFS order 10 • Tree DP 10 • Diameter 10 • LCA and binary lifting 11 • Rerooting 11 • Strongly connected components 11 • Euler path 11 • Bipartite check 12 • Bipartite matching 12

9 Math and number theory **12**

Integer pitfalls 12 • gcd, lcm and fractions 12 • Extended Euclid: $ax + by = c$ 12 • Modular arithmetic 13 • Formulas and numbers 13 • Sieve of Eratosthenes 13 • Factorising one number 13 • Divisors 13 • Chinese remainder theorem 13 • Permutation cycles 14 • Matrix power 14 • XOR tricks 14 • Big numbers 14

10 Counting and probability **15**

Counting formulas 15 • Binomial coefficients mod p 15 • Pascal's triangle 15 • Inclusion–exclusion 15 • Probability rules 15 • Expected-value DP 15 • Probability DP 15 • Answers as a fraction mod p 15 • Doubles 15

11 Games **16**

Winning and losing 16 • Win / lose table 16 • Nim and Grundy numbers 16 • Taking from both ends 16

12 Geometry **16**

Rules 16 • Point and cross product 16 • Segment intersection 16 • Lines and distances 16 • Polygon area, point in polygon 17 • What shape is it? 17 • Convex hull 17 • Closest pair of points 17 • Angles and rotation 17 • Circles 17 • Formulas 18

13 Strings **18**

Which string tool? 18 • Counting letters 18 • Palindromes 18 • KMP 18 • Rolling hash 18 • Trie 19 • Palindrome by rearranging 19

14 Brute force and testing **19**

All subsets 19 • All orders 19 • Backtracking with pruning 19 • Stress test 19 • Interactive problems 20 • Debugging 20

Notes **21**

1 Contest

1.1 How big can n be?

About 10^8 simple steps per second.

$n \leq$	time	typical idea
10	$O(n!)$	every order (`next_permutation`)
20	$O(2^n \cdot n)$	every subset, bitmask DP
500	$O(n^3)$	three loops, Floyd–Warshall
5 000	$O(n^2)$	two loops, $n \times n$ DP
10^6	$O(n \log n)$	sort, binary search, Dijkstra, set/map
10^8	$O(n)$	one loop, prefix sums
10^{18}	$O(\log n)$	formula, binary search on the answer

`int` $\approx 2.1 \cdot 10^9$, `long long` $\approx 9.2 \cdot 10^{18}$, `double` ≈ 15 digits.

1.2 Which technique?

the statement says	try	p.
fewest moves / steps, maze	BFS, grid BFS	8
cheapest route, weighted roads	Dijkstra (shortest path table)	8
"smallest x such that it works"	binary search on the answer	5
best total over choices, $n \leq 5000$	DP	6
number of ways (mod $10^9 + 7$)	DP, counting formulas	15
choose items, weight limit	knapsack	6
$n \leq 20$	all subsets, bitmask DP	19
$n \leq 10$, find the best order	all orders	19
intervals, meetings, schedules	sort by end, greedy	4
rooms needed at the same time	sweep $+1/-1$	4
sort and take, pair up	greedy moves	4
"connected?", groups merge	Union-Find	10
"a before b", dependencies	topological sort	9
connect everything cheaply	Kruskal	10
tree, distances between nodes	tree facts, LCA	10
directed cycles, "reach each other"	SCC	11
same step repeated 10^{18} times	binary lifting, cycles	11
sum of a range, many queries	prefix sums	5
add to a range, many times	difference array	5
best window / subarray	two pointers	5
nearest bigger to the left / right	monotonic stack	5
up then down, few questions	peak search	5
"interactive"	flush, binary search	20
"modulo $10^9 + 7$"	modular arithmetic	13
repeats every a and every b	gcd / lcm, CRT	12
primes, divisors, factors	sieve, factorising	13
recurrence, n up to 10^{18}	matrix power	14
digits of numbers up to 10^{18}	digit DP	8
hundreds of digits	big numbers, Python	14
probability, expected value	expected-value DP	15
two players, optimal play	games	16
substrings, patterns, words	string tool table	18
points, lines, polygons, angles	geometry rules	16
clocks, dates	time and dates	3
rotate / mirror a grid	grids	3
text with commas, odd input	splitting text	3

1.3 Template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;
#define all(x) begin(x), end(x)
#define rep(i, a, b) for (int i = (a); i < (b); i++)
const ll INF = 1e18;

// dbg(x) prints "x = ..." to cerr (also pairs, vectors), but only
// with -DLOCAL (the c command). On the judge it does nothing.
#ifdef LOCAL
template <class A, class B>
ostream& operator<<(ostream& os, const pair<A, B>& p) {
    return os << '(' << p.first << ", " << p.second << ')';
}
template <class T>
ostream& operator<<(ostream& os, const vector<T>& v) {
    os << '{';
    rep(i, 0, (int)v.size()) os << (i ? ", " : "") << v[i];
    return os << '}';
}
#define dbg(x) cerr << #x << " = " << (x) << endl
#else
#define dbg(x)
#endif

int main() {
    cin.tie(0)->sync_with_stdio(0);     // fast input/output
    cout << fixed << setprecision(10);  // 10 decimals
    int n;
    cin >> n;
    cout << n << '\n';   // '\n', not endl: endl is slow
}
```

1.4 Terminal setup

```
# Once at the start: save the template as ~/template.cpp, paste this
# into the terminal (or put it in ~/.bashrc). Then for problem A:
#   mk a     folder a/ with a/a.cpp copied from the template
#   c a      compile a/a.cpp: dbg on, checks for out-of-range
#   r a      compile and run: paste the input, then Ctrl-D
#   r a 1    compile and run on a/1.in
mk() { mkdir -p "$1" && cp -n ~/template.cpp "$1/$1.cpp"; }
c() {
    g++ -std=c++17 -O2 -Wall -g -DLOCAL \
        -fsanitize=address,undefined "$1/$1.cpp" -o "$1/$1"
}
r() {
    c "$1" || return
    if [ -n "$2" ]; then ./"$1/$1" < "$1/$2.in"
    else ./"$1/$1"
    fi
}
```

1.5 Reading input

```cpp
string line;
getline(cin >> ws, line);  // whole line; ws skips the old newline
stringstream ss(line);     // split that line into words
string word;
while (ss >> word) { /* use word */ }
int x;
while (cin >> x) { /* ... */ }  // read numbers until input ends
```

2 C++ quick reference

2.1 Sorting

```cpp
vi v = {5, 1, 4, 1};
sort(all(v));                       // 1 1 4 5
sort(all(v), greater<>());          // 5 4 1 1  (biggest first)
v.erase(unique(all(v)), v.end());  // drop repeats (sort first)
// pairs sort by .first, then .second. Own order with a lambda:
vector<pii> ps = {{1, 9}, {2, 3}, {3, 3}};
sort(all(ps), [](const pii& a, const pii& b) {
    if (a.second != b.second) return a.second < b.second;
    return a.first > b.first;       // tie: bigger .first first
});                                 // {3,3} {2,3} {1,9}
struct Job { int start, end; };
vector<Job> jobs = {{1, 5}, {0, 5}, {2, 3}};
sort(all(jobs), [](const Job& a, const Job& b) {  // end, then start
    return tie(a.end, a.start) < tie(b.end, b.start);
});
// a comparator must be strict: use <, never <= (it can crash)
```

2.2 Useful algorithms

```cpp
vi w = {3, 9, 2, 9};
int mx = *max_element(all(w));              // 9
int pos = max_element(all(w)) - w.begin();  // 1 (the first max)
ll total = accumulate(all(w), 0LL);         // 0LL: sum as long long
int nines = count(all(w), 9);               // 2
bool has2 = find(all(w), 2) != w.end();     // true
int low = min({4, 7, 1});                   // min of many: braces
vi idx(4);
iota(all(idx), 0);                          // 0 1 2 3
int orders = 0;                             // every order, if sorted
do orders++; while (next_permutation(all(idx)));  // 24
ll g = gcd(12LL, 18LL), l = lcm(4LL, 6LL);  // 6 and 12
```

2.3 Containers

Sorted / next bigger: `set`, `map` ($\log n$). Lookup only: `unordered_map`. Always the min or max: `priority_queue`. Both ends: `deque`.

```cpp
set<int> s = {5, 1, 8};             // sorted, no duplicates
s.insert(3);
s.erase(8);
bool has3 = s.count(3);             // count is 0 or 1
auto it = s.lower_bound(4);         // first element >= 4: 5
int first = *s.begin(), last = *s.rbegin();   // 1 and 5
multiset<int> ms = {2, 2, 7};       // like set, keeps duplicates
ms.erase(ms.find(2));               // removes ONE 2; erase(2): all
map<string, int> cnt;
cnt["apple"]++;                     // a new key starts at 0
bool pear = cnt.count("pear");      // check without adding it
for (auto& [key, val] : cnt) {}     // loops in sorted key order
priority_queue<int> maxpq;                  // biggest on top
priority_queue<int, vi, greater<>> minpq;   // smallest on top
maxpq.push(4); maxpq.push(9); minpq.push(4); minpq.push(9);
int big = maxpq.top(), small = minpq.top();   // 9 and 4
deque<int> dq = {1, 2};
dq.push_front(0); dq.pop_back();    // 0 1
vector<vector<int>> grid(3, vector<int>(4, 0));   // 3 x 4 of zeros
```

2.4 Strings and characters

```cpp
string s = "Hello World";
string w = s.substr(6, 5);              // "World": start, length
size_t at = s.find("lo");               // 3, or string::npos
bool found = s.find("xyz") != string::npos;   // false
int x = stoi("42");
ll y = stoll("12345678901");
double d = stod("3.5");
string t = to_string(17);
char ch = 'e';
int k = ch - 'a';                       // 'a' -> 0, ..., 'z' -> 25
char back = 'a' + k;                    // 'e'
bool dig = isdigit('7'), let = isalpha('x');
for (char& c : s) c = tolower(c);       // "hello world"
string stars(3, '*');                   // "***"
string r = s;
reverse(all(r));                        // "dlrow olleh"
```

2.5 Bits

```cpp
ll mask = 0b10110;                          // bits 1, 2, 4 set
int ones = __builtin_popcountll(mask);      // 3
int lowest = __builtin_ctzll(mask);         // 1   (mask must be > 0)
int highest = 63 - __builtin_clzll(mask);   // 4   (mask must be > 0)
bool bit2 = mask >> 2 & 1;                  // is bit 2 set? true
mask |= 1LL << 5;                           // set bit 5
mask &= ~(1LL << 1);                        // clear bit 1
mask ^= 1;                                  // flip bit 0
int n = 3, subsets = 0;
rep(sub, 0, 1 << n) subsets++;              // all 8 subsets
int m = 0b101, parts = 0;                   // every non-empty submask
for (int sub = m; sub > 0; sub = (sub - 1) & m) parts++;  // 3
```

2.6 Lambdas

```
auto sq = [](ll v) { return v * v; };
int calls = 0;
auto bump = [&]() { calls++; };   // [&]: can change outer vars
bump(); bump();                   // calls == 2
// recursive lambda: give it itself as the first argument
auto fib = [&](auto&& self, int i) -> ll {
    return i < 2 ? i : self(self, i - 1) + self(self, i - 2);
};
ll f10 = fib(fib, 10);            // 55
// or: function<ll(int)> f = [&](int i) -> ll { ... f(i - 1) ... };
```

2.7 Ordered set $O(\log n)$

The k-th smallest, or how many are below x, while inserting and erasing.

```
// set with "k-th smallest" and "how many are < x", both O(log n)
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using oset = tree<T, null_type, less<T>, rb_tree_tag,
                  tree_order_statistics_node_update>;
// oset<int> s; s.insert(5); s.erase(5);
// *s.find_by_order(k): k-th smallest, k from 0
// s.order_of_key(x):   how many elements are < x
// duplicates: store pairs {value, unique index}
```

2.8 Hash for pair keys $O(1)$

unordered_map with pair keys does not compile without it.

```
// unordered_map / unordered_set with pair keys need a hash
// (map<pii, int> works without one, in O(log n))
struct PairHash {
    size_t operator()(const pair<ll, ll>& p) const {
        unsigned long long x = p.first * 1000000007ULL + p.second;
        x ^= x >> 31;
        x *= 0x9e3779b97f4a7c15ULL;
        return x ^ (x >> 29);
    }
};
// unordered_map<pair<ll, ll>, int, PairHash> seen;
```

3 Simulation and parsing

3.1 Grids $O(R \cdot C)$

Rotate, transpose, mirror; 8 neighbours; wrap around the edges.

```
// read: vector<string> g(R); for (auto& row : g) cin >> row;
// rotate 90 degrees clockwise: an R x C grid becomes C x R
vector<string> rotateCW(const vector<string>& g) {
    int R = g.size(), C = g[0].size();
    vector<string> h(C, string(R, ' '));
    rep(r, 0, R) rep(c, 0, C) h[c][R - 1 - r] = g[r][c];
    return h;
}
// swap rows and columns: h[c][r] = g[r][c]
vector<string> transpose(const vector<string>& g) {
    int R = g.size(), C = g[0].size();
    vector<string> h(C, string(R, ' '));
    rep(r, 0, R) rep(c, 0, C) h[c][r] = g[r][c];
    return h;
}
// mirror left-right: for (auto& row : g) reverse(all(row));
// upside down: reverse(all(g));   counter-clockwise: rotateCW 3x
// 8 neighbours (with the diagonals):
int dr8[] = {-1, -1, -1, 0, 0, 1, 1, 1};
int dc8[] = {-1, 0, 1, -1, 1, -1, 0, 1};
// wrap around the edges (torus): nr = (r + dr8[d] + R) % R
```

3.2 Time and dates $O(1)$

Clock times as minutes, leap years, days between dates, day of the week.

```
// "HH:MM" -> minutes since midnight, and back
int toMin(const string& t) {
    return stoi(t.substr(0, 2)) * 60 + stoi(t.substr(3, 2));
}
string toTime(int m) {   // 0 <= m < 1440
    char buf[16];
    snprintf(buf, sizeof buf, "%02d:%02d", m / 60, m % 60);
    return buf;
}
// minutes from a to b, also past midnight: (b - a + 1440) % 1440
bool leap(int y) {
    return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
}
int daysIn(int m, int y) {   // month m = 1..12
    static const int d[] = {31, 28, 31, 30, 31, 30,
                            31, 31, 30, 31, 30, 31};
    return d[m - 1] + (m == 2 && leap(y));
}
// a day number: days between two dates = dayNum(b) - dayNum(a)
ll dayNum(ll y, ll m, ll d) {
    if (m < 3) y--, m += 12;
    return 365 * y + y / 4 - y / 100 + y / 400 + (153 * m - 457) / 5
           + d - 306;
}
// day of the week: 0 = Monday, ..., 6 = Sunday
int weekday(ll y, ll m, ll d) { return (dayNum(y, m, d) + 6) % 7; }
```

3.3 Splitting text $O(n)$

Fields separated by commas or other characters; numbers hidden in text.

```
// split "a,b,,c" at ',' -> {"a", "b", "", "c"}
vector<string> split(const string& s, char sep) {
    vector<string> parts(1);
    for (char c : s) {
        if (c == sep) parts.push_back("");
        else parts.back() += c;
    }
    return parts;
}
// the numbers inside text: "12:30-14:05" -> {12, 30, 14, 5}.
// Minus signs are dropped
vi numbersIn(string s) {
    for (char& c : s) if (!isdigit(c)) c = ' ';
    stringstream ss(s);
    vi nums;
    int x;
    while (ss >> x) nums.push_back(x);
    return nums;
}
```

3.4 Number bases $O(\log x)$

Binary, hex, base 36 and back.

```
// x >= 0 written in base b (2..36), digits 0-9 then a-z
string toBase(ll x, int b) {
    if (x == 0) return "0";
    string s;
    while (x > 0) {
        int d = x % b;
        s += d < 10 ? '0' + d : 'a' + d - 10;
        x /= b;
    }
    reverse(all(s));
    return s;
}
ll fromBase(const string& s, int b) {
    ll x = 0;
    for (char c : s)
        x = x * b + (isdigit(c) ? c - '0' : tolower(c) - 'a' + 10);
    return x;
}
// base 2 only: bitset<64>(x).to_string(), stoll(s, nullptr, 2)
```

4 Sorting and greedy

4.1 Greedy moves

- Sort, then take the biggest (or smallest) first.
- Pair the smallest with the largest.
- End every piece as early as possible.
- Give each person the smallest thing that is big enough.
- Not sure? Break it with 2–3 tiny cases or a brute force. Fails and n small: DP.

4.2 Finding the sort order $O(n \log n)$

Compare the cost of "a then b" with "b then a". Below: a first iff $t_a w_b < t_b w_a$. Biggest glued number: sort strings by `a + b > b + a`.

```
// one machine does the jobs {t, w} one after another; minimise the
// sum of w * (finish time). All t, w > 0. a goes before b if
// t_a / w_a < t_b / w_b; cross-multiplied: exact, no doubles
sort(all(jobs), [](auto& a, auto& b) {
    return a.first * b.second < b.first * a.second;
});
ll now = 0, cost = 0;
for (auto [t, w] : jobs) { now += t; cost += w * now; }
```

4.3 Most non-overlapping intervals $O(n \log n)$

Take the interval that ends first. Fewest points hitting every closed $[s, e]$: the same code with end $e + 1$.

```
// most intervals [s, e) with no two overlapping (touching is ok).
// Take the one that ends first, skip all that start before its end
int maxDisjoint(vector<pair<ll, ll>> iv) {  // iv[i] = {s, e}
    sort(all(iv), [](auto& a, auto& b) {
        return a.second < b.second;
    });
    int cnt = 0;
    ll last = -INF;  // end of the last interval we took
    for (auto [s, e] : iv)
        if (s >= last) { cnt++; last = e; }
    return cnt;
}
```

4.4 Merging intervals $O(n \log n)$

Union of intervals: length covered, gaps, "is x covered?".

```
// union of intervals [s, e]: sorted, no overlaps. Intervals that
// touch ([1,3] and [3,5]) are merged too; < keeps them apart
vector<pair<ll, ll>> mergeIntervals(vector<pair<ll, ll>> iv) {
    sort(all(iv));
    vector<pair<ll, ll>> res;
    for (auto [s, e] : iv) {
        if (!res.empty() && s <= res.back().second)  // overlaps
            res.back().second = max(res.back().second, e);
        else res.push_back({s, e});
    }
    return res;
}
// length covered: add up e - s over the result
```

4.5 Most intervals at the same time $O(n \log n)$

Rooms or machines needed at once: $+1$ at a start, -1 at an end, sort, running count.

```
// most intervals [s, e) open at the same moment = rooms needed.
// Events: +1 at s, -1 at e. At equal times {t, -1} sorts before
// {t, +1}, so [1,3) and [3,5) can share a room
int maxOverlap(const vector<pair<ll, ll>>& iv) {
    vector<pair<ll, int>> ev;
    for (auto [s, e] : iv) {
        ev.push_back({s, +1});
        ev.push_back({e, -1});
    }
    sort(all(ev));
    int cur = 0, best = 0;
    for (auto [t, d] : ev) {
        cur += d;
        best = max(best, cur);
    }
    return best;
}
```

4.6 Jobs with deadlines $O(n \log n)$

Most profit or most jobs in time: by deadline, take every job; when they no longer fit, drop the worst one taken.

```
// jobs {d, p}: each takes 1 time unit and earns p >= 0 if it is
// done by time d (slots 1, 2, ..). Most profit: go by deadline,
// keep the jobs taken in a min-heap, drop the cheapest if too many
ll maxProfit(vector<pair<ll, ll>> jobs) {
    sort(all(jobs));
    priority_queue<ll, vector<ll>, greater<>> pq;  // min on top
    ll sum = 0;
    for (auto [d, p] : jobs) {
        pq.push(p); sum += p;
        if ((ll)pq.size() > d) { sum -= pq.top(); pq.pop(); }
    }
    return sum;
}
// jobs {d, len}: one at a time from time 0, must end by time d.
// Most jobs done: go by deadline; if we are late, drop the
// longest job taken so far
int maxJobs(vector<pair<ll, ll>> jobs) {
    sort(all(jobs));
    priority_queue<ll> pq;  // largest on top
    ll now = 0;
    for (auto [d, len] : jobs) {
        pq.push(len); now += len;
        if (now > d) { now -= pq.top(); pq.pop(); }
    }
    return pq.size();
}
```

4.7 Matching with the smallest that fits $O((n + m) \log m)$

Person i accepts sizes in $[lo_i, hi_i]$, most pairs: smallest hi first, give it the smallest item $\geq lo$. Only "at least lo": $hi =$ `INF`.

```
// person i accepts an item of size lo..hi; each item goes to at
// most one person. Most people served: smallest hi first, gets
// the smallest free item >= lo. people[i] = {hi, lo}
int maxMatch(vector<pair<ll, ll>> people, vector<ll> items) {
    sort(all(people));  // smallest hi first
    multiset<ll> avail(all(items));
    int cnt = 0;
    for (auto [hi, lo] : people) {
        auto it = avail.lower_bound(lo);
        if (it != avail.end() && *it <= hi) {
            avail.erase(it);  // erase(it), not erase(value)!
            cnt++;
        }
    }
    return cnt;
}
```

4.8 The answer is one of the input values $O(n \log n)$

Choose one number (price, limit, time): try every input value (and ± 1, 0). Smallest $\sum |x - a_i|$: x is the median.

```
// customer i buys if the price is <= b[i]. One price p for all:
// earn p * (number of b[i] >= p). The best p is one of the b[i]
// (raise p up to the next b[i]: same buyers, more money)
sort(all(b), greater<>());  // biggest first
ll best = 0;
rep(i, 0, n) best = max(best, b[i] * (i + 1));  // i+1 buyers
```

5 Searching and arrays

5.1 Binary search on the answer $O(\log(hi - lo))$ checks

"Smallest t such that ..." and checking one t is easy: works for $t \Rightarrow$ works for $t + 1$. Write `ok(t)`.

```
// smallest x in [lo, hi] with ok(x) true; ok looks like F..F T..T.
// returns hi + 1 if ok is false everywhere
template <class F> ll firstTrue(ll lo, ll hi, F ok) {
    hi++;
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
// use: ll ans = firstTrue(0, 2e9, [&](ll t) { return canDo(t); });
// largest x with ok(x), where ok looks like T..T F..F:
//        firstTrue(lo, hi, [&](ll x) { return !ok(x); }) - 1
```

5.2 Binary search on real numbers 100 steps

The answer is a real number and ok is monotone.

```
// real-valued answer: halve the range 100 times (plenty of digits)
double lo = 0, hi = 1e9;
rep(it, 0, 100) {
    double mid = (lo + hi) / 2;
    if (ok(mid)) hi = mid;
    else lo = mid;
}
// answer = hi
```

5.3 Finding a peak $O(\log n)$

Goes up then down (or down then up), find the top. Often interactive. Integers: binary search on "is the next value lower?"; reals: ternary search.

```
// integers in [lo, hi], f goes up then down (no flat parts!):
// the peak is the first x whose next value is lower. Needs firstTrue
ll peak = firstTrue(lo, hi - 1,
                    [&](ll x) { return f(x) > f(x + 1); });
```

```
// real numbers, f goes up then down: returns x of the maximum
// (for a minimum, use -f)
template <class F> double ternaryMax(double lo, double hi, F f) {
    rep(it, 0, 200) {
        double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1;  // peak is right of m1
        else hi = m2;                // peak is left of m2
    }
    return lo;
}
```

5.4 Searching in sorted data

```
// v sorted:
//   first index with v[i] >= x:  lower_bound(all(v), x) - v.begin()
//   first index with v[i] >  x:  upper_bound(all(v), x) - v.begin()
//   how often x occurs:  upper_bound(...) - lower_bound(...)
// set<ll> s:
//   first element >= x:  s.lower_bound(x)   (s.end() if none)
//   last element <= x:   auto it = s.upper_bound(x);
//                        if (it != s.begin()) { --it; use *it; }
```

5.5 Prefix sums $O(1)$ per range

Sum of many ranges; 2D: sum of any rectangle.

```
// pre[i] = a[0] + ... + a[i-1]
// sum of a[l..r] (inclusive) = pre[r + 1] - pre[l]
vector<ll> pre(n + 1, 0);
rep(i, 0, n) pre[i + 1] = pre[i] + a[i];
```

```
// P[i][j] = sum of g[0..i-1][0..j-1]  (g has R rows, C columns)
vector<vector<ll>> P(R + 1, vector<ll>(C + 1, 0));
rep(i, 0, R) rep(j, 0, C)
    P[i+1][j+1] = g[i][j] + P[i][j+1] + P[i+1][j] - P[i][j];
// sum of the rectangle rows r1..r2, columns c1..c2 (inclusive)
auto rect = [&](int r1, int c1, int r2, int c2) {
    return P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1];
};
```

5.6 Difference array $O(1)$ per update

Many "add v to $a[l..r]$", the final values only at the end.

```
// many updates "add v to a[l..r]", then read every value once.
// O(1) per update, O(n) at the end
vector<ll> diff(n + 1, 0);
for (auto [l, r, v] : updates) { diff[l] += v; diff[r + 1] -= v; }
ll cur = 0;
rep(i, 0, n) { cur += diff[i]; a[i] += cur; }
```

5.7 Two pointers / sliding window $O(n)$

Best subarray with a rule that stays true when the window shrinks (sum $\leq K$ with values ≥ 0, at most k different letters).

```
// longest subarray with sum <= K (all a[i] >= 0). The window is
// a[l..r]: r moves right, l catches up while the sum is too big.
// O(n): every index enters and leaves the window once
int best = 0;
ll sum = 0;
for (int l = 0, r = 0; r < n; r++) {
    sum += a[r];
    while (sum > K) sum -= a[l++];
    best = max(best, r - l + 1);
}
```

5.8 Coordinate compression $O(n \log n)$

Huge values (10^9), few of them: replace by ranks to use as indices.

```
// replace values by their rank 0..k-1 among the distinct values,
// keeping their order (big values -> small array indices)
vector<ll> vals(all(a));
sort(all(vals));
vals.erase(unique(all(vals)), vals.end());   // k = vals.size()
vi id(n);
rep(i, 0, n) id[i] = lower_bound(all(vals), a[i]) - vals.begin();
// original value back: vals[id[i]]
```

5.9 Monotonic stack $O(n)$

Nearest bigger element to the right. Left side: loop backwards. Smaller: flip the comparison.

```
// nxt[i] = first index j > i with a[j] > a[i], or n if none.
// The stack holds indices still waiting for their answer; their
// values go down from bottom to top. O(n) in total
vi nextGreater(const vector<ll>& a) {
    int n = a.size();
    vi nxt(n, n), st;
    rep(i, 0, n) {
        while (!st.empty() && a[st.back()] < a[i]) {
            nxt[st.back()] = i;  // a[i] answers st.back()
            st.pop_back();
        }
        st.push_back(i);
    }
    return nxt;
}
```

5.10 Largest rectangle in a histogram $O(n)$

Largest free rectangle in a grid: per row, $h[c]$ = free cells straight above; run on every row, $O(R \cdot C)$.

```
// largest rectangle under a histogram with bar heights h. O(n).
// When bar i is popped, it is the lowest bar of a rectangle that
// stretches from just after the new stack top up to i - 1
ll largestRect(const vector<ll>& h) {
    int n = h.size();
    vi st;   // indices, heights going up from bottom to top
    ll best = 0;
    rep(i, 0, n + 1) {
        ll cur = i < n ? h[i] : 0;   // height 0 at the end: pop all
        while (!st.empty() && h[st.back()] >= cur) {
            ll height = h[st.back()];
            st.pop_back();
            int left = st.empty() ? 0 : st.back() + 1;
            best = max(best, height * (i - left));
        }
        st.push_back(i);
    }
    return best;
}
```

5.11 Sliding window minimum $O(n)$

Min (or max) of every window of k, e.g. `dp[i] = min(dp[i-k..i-1]) + c`.

```
// mn[j] = min of the window a[j..j+k-1], for every j. O(n).
// The deque holds indices; their values go up from front to back
vector<ll> slidingMin(const vector<ll>& a, int k) {
    vector<ll> mn;
    deque<int> dq;
    rep(i, 0, (int)a.size()) {
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();  // out of window
        if (i >= k - 1) mn.push_back(a[dq.front()]);
    }
    return mn;
}
// window maximum: flip >= to <=
```

6 Dynamic programming

6.1 How to build a DP

- **State:** what you must know to finish the rest. `dp[i]`; `dp[i][j]` (two strings, a grid); `dp[l][r]` (a piece); add weight used, last item, speed...
- **Transition:** try every last choice: `dp[i] = best(cost + dp[smaller])`.
- **Base:** `dp[0] = 0`. Impossible: `INF` (min), `-INF` (max), 0 (counting).
- **Order:** fill a state after every state it reads, or memoize.
- **Fast enough:** states $\times$ choices $\leq 10^8$. Row i only reads row $i-1$: keep two rows.
- **The choices:** store `from[state]`, walk back. **Counting:** `% MOD` after every addition.
- **Games:** winning iff some move leads to a losing position.

6.2 Memoized recursion states $\times$ choices

The recursion is easy but solves the same subproblem again and again: store each answer.

```
// memoized recursion. Example: from stone i you may jump to
// i+1 .. i+k, paying |h[i] - h[j]|. Cheapest way from stone 0 to
// stone n-1. memo[i] == -1 means "not computed yet"
int n, k;
vector<ll> h, memo;
ll solve(int i) {  // cheapest cost from stone i to the last one
    if (i == n - 1) return 0;           // base case
    ll& res = memo[i];  // a reference: setting res fills memo[i]
    if (res != -1) return res;          // computed before
    res = INF;
    rep(j, i + 1, min(n, i + k + 1))
        res = min(res, abs(h[i] - h[j]) + solve(j));
    return res;
}
// main: read n, k, h;  memo.assign(n, -1);  cout << solve(0);
```

6.3 Best subarray sum $O(n)$

Largest sum of consecutive elements. Smallest: flip the signs.

```
// largest sum of a block a[l..r] of consecutive elements, not
// empty (if the empty block counts, start with best = 0).
// cur = largest sum of a block that ends at the current x
ll best = -INF, cur = 0;
for (ll x : a) {
    cur = max(x, cur + x);  // extend the block, or start anew at x
    best = max(best, cur);
}
```

6.4 0/1 knapsack and subset sum $O(n \cdot W)$

Each item at most once, weight $\leq W$, most value. Subset sum: which totals can be made (bitset: 64$\times$ faster). Two halves as equal as possible: the largest $s \leq total/2$ with `can[s]`.

```
// 0/1 knapsack: items {w, v}, each used at most once, total weight
// <= W. dp[c] = most value with weight <= c. O(n * W)
ll knapsack(const vector<pair<int, ll>>& items, int W) {
    vector<ll> dp(W + 1, 0);
    for (auto [w, v] : items)
        for (int c = W; c >= w; c--)  // backwards: item used once
            dp[c] = max(dp[c], dp[c - w] + v);
    return dp[W];
}
```

```
// which sums can a subset of w make? can[s] == 1 if some subset
// has sum s. O(n * S / 64): fast up to S = 10^6 and more
bitset<100001> can;  // size = largest sum + 1, must be a constant
can[0] = 1;
for (int x : w) can |= can << x;
```

6.5 Coin change $O(n \cdot S)$

Every coin as often as you like: fewest coins, number of ways. Greedy fails: coins 1, 3, 4 for 6.

```
// coins (different values) may be used any number of times.
// fewest[s] = fewest coins with sum exactly s (INF: impossible)
// ways[s]  = ways to pay s, 1+2 and 2+1 count once (mod 1e9+7)
// order[s] = ways to pay s, 1+2 and 2+1 count twice. O(n * S)
const ll MOD = 1e9 + 7;
vector<ll> fewest(S + 1, INF), ways(S + 1, 0), order(S + 1, 0);
fewest[0] = 0; ways[0] = order[0] = 1;
for (int c : coins)  // coins outside
    for (int s = c; s <= S; s++) {  // forwards: c can be reused
        fewest[s] = min(fewest[s], fewest[s - c] + 1);
        ways[s] = (ways[s] + ways[s - c]) % MOD;
    }
rep(s, 1, S + 1) for (int c : coins)  // sums outside
    if (c <= s) order[s] = (order[s] + order[s - c]) % MOD;
```

6.6 Longest increasing subsequence $O(n \log n)$

Fewest deletions to make it sorted: $n - \mathrm{LIS}$. Boxes in boxes: sort by the first value, ties by the second descending, then LIS on the second.

```
// longest strictly increasing subsequence in O(n log n); returns
// its indices. d[k] = smallest last value of an increasing
// subsequence of length k+1 seen so far (d itself is no answer)
vi lis(const vector<ll>& a) {
    vector<ll> d;
    vi at, prv(a.size(), -1);  // at[k]: index of d[k]
    rep(i, 0, (int)a.size()) {
        // non-decreasing (equal allowed): upper_bound here
        int k = lower_bound(all(d), a[i]) - d.begin();
        if (k == (int)d.size()) d.push_back(a[i]), at.push_back(i);
        else d[k] = a[i], at[k] = i;
        if (k > 0) prv[i] = at[k - 1];  // element before a[i]
    }
    vi res;
    for (int i = at.empty() ? -1 : at.back(); i != -1; i = prv[i])
        res.push_back(i);
    reverse(all(res));
    return res;
}
```

6.7 Longest common subsequence $O(n \cdot m)$

The longest subsequence two strings (or arrays) share.

```
// longest common subsequence of a and b, O(|a| * |b|).
// dp[i][j] = LCS length of the first i chars of a, j chars of b
string lcs(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vi> dp(n + 1, vi(m + 1, 0));
    rep(i, 0, n) rep(j, 0, m)
        dp[i + 1][j + 1] = a[i] == b[j] ? dp[i][j] + 1
                         : max(dp[i][j + 1], dp[i + 1][j]);
    string res;  // walk back from dp[n][m] to find one LCS
    for (int i = n, j = m; i > 0 && j > 0;) {
        if (a[i - 1] == b[j - 1]) res += a[i - 1], i--, j--;
        else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
        else j--;
    }
    reverse(all(res));
    return res;  // length: dp[n][m]
}
```

6.8 Edit distance $O(n \cdot m)$

Fewest insertions, deletions and replacements that turn a into b.

```
// fewest single-char inserts, deletes and replaces to turn a into
// b. dp[i][j] = cost for the first i chars of a -> first j of b
int editDistance(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vi> dp(n + 1, vi(m + 1));
    rep(i, 0, n + 1) dp[i][0] = i;  // delete all i chars
    rep(j, 0, m + 1) dp[0][j] = j;  // insert all j chars
    rep(i, 1, n + 1) rep(j, 1, m + 1)
        dp[i][j] = min({dp[i - 1][j] + 1,   // delete a[i-1]
                        dp[i][j - 1] + 1,   // insert b[j-1]
                        dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])});
    return dp[n][m];
}
```

6.9 Paths in a grid $O(R \cdot C)$

Only moves right or down: count paths, most coins. Moves in every direction: BFS or Dijkstra.

```
// paths from the top left to cell (r, c), moving only right or
// down and never onto '#'. g has R rows and C columns
const ll MOD = 1e9 + 7;
vector<vector<ll>> ways(R, vector<ll>(C, 0));
rep(r, 0, R) rep(c, 0, C) {
    if (g[r][c] == '#') continue;
    if (r == 0 && c == 0) { ways[r][c] = 1; continue; }
    if (r > 0) ways[r][c] += ways[r - 1][c];  // came from above
    if (c > 0) ways[r][c] += ways[r][c - 1];  // came from the left
    ways[r][c] %= MOD;
}
// answer: ways[R - 1][C - 1]
```

6.10 Cutting a string into words $O(|s| \cdot L)$

Split a string into words from a list: possible? how many ways? fewest pieces (`min` instead of `+`)?

```
// ways to cut s into words from the list (mod 1e9+7).
// ways[i] = ways to cut the first i characters; the last piece
// is s[i-len .. i-1]. O(|s| * L) lookups, L = longest word
ll wordBreak(const string& s, const vector<string>& words) {
    const ll MOD = 1e9 + 7;
    set<string> dict(all(words));
    int n = s.size(), L = 0;
    for (auto& w : words) L = max(L, (int)w.size());
    vector<ll> ways(n + 1, 0);
    ways[0] = 1;  // the empty prefix: one way
    rep(i, 1, n + 1)
        for (int len = 1; len <= min(i, L); len++)
            if (dict.count(s.substr(i - len, len)))
                ways[i] = (ways[i] + ways[i - len]) % MOD;
    return ways[n];
}
```

6.11 Cutting an array into k parts $O(k \cdot n^2)$

Consecutive groups, each with a cost: cheapest total. Any number of groups: drop j.

```
// split a into exactly k consecutive non-empty parts; minimise the
// total cost of the parts, here cost = (sum of the part)^2.
// dp[j][i] = best for the first i elements cut into j parts
ll splitK(const vector<ll>& a, int k) {
    int n = a.size();
    vector<ll> pre(n + 1, 0);
    rep(i, 0, n) pre[i + 1] = pre[i] + a[i];
    vector<vector<ll>> dp(k + 1, vector<ll>(n + 1, INF));
    dp[0][0] = 0;
    rep(j, 1, k + 1) rep(i, 1, n + 1)
        rep(p, 0, i) if (dp[j - 1][p] < INF) {  // last: a[p..i-1]
            ll s = pre[i] - pre[p];
            dp[j][i] = min(dp[j][i], dp[j - 1][p] + s * s);
        }
    return dp[k][n];  // INF if k > n
}
```

6.12 Range sums inside a DP $O(n)$

`dp[i]` adds up a range of earlier states: keep a prefix sum of dp.

```
// ways to climb n stairs taking 1..k stairs per step (mod 1e9+7).
// dp[i] = dp[i-k] + ... + dp[i-1] is a range sum, so keep
// pre[i] = dp[0] + ... + dp[i-1]: O(n) instead of O(n * k)
const ll MOD = 1e9 + 7;
vector<ll> dp(n + 1, 0), pre(n + 2, 0);
dp[0] = 1; pre[1] = 1;
rep(i, 1, n + 1) {
    dp[i] = (pre[i] - pre[max(0, i - k)] + MOD) % MOD;
    pre[i + 1] = (pre[i] + dp[i]) % MOD;
}
```

6.13 Weighted intervals $O(n \log n)$

Non-overlapping intervals with the most total value. A pass from day d covering L days: the next state is `lower_bound`$(d + L)$.

```
// jobs {e, s, p}: run from s to e (s < e) and earn p. Most profit
// with no two jobs overlapping (touching is ok). O(n log n).
// dp[i] = most profit using only the first i jobs (by end time)
ll bestProfit(vector<array<ll, 3>> jobs) {
    sort(all(jobs));
    int n = jobs.size();
    vector<ll> ends(n), dp(n + 1, 0);
    rep(i, 0, n) ends[i] = jobs[i][0];
    rep(i, 0, n) {
        auto [e, s, p] = jobs[i];
        // jobs 0..j-1 end at or before s, so they fit before job i
        int j = upper_bound(all(ends), s) - ends.begin();
        dp[i + 1] = max(dp[i], dp[j] + p);  // skip i, or take it
    }
    return dp[n];
}
```

6.14 Interval DP $O(n^3)$

Merging neighbours, cutting a stick, brackets, taking from both ends: `dp[l][r]`, try every split point. $n \leq 400$.

```
// n piles in a row; merging two neighbouring piles costs their
// total size. Cheapest way to merge everything. O(n^3).
// dp[l][r] = cheapest way to merge piles l..r into one
ll mergePiles(const vector<ll>& a) {
    int n = a.size();
    vector<ll> pre(n + 1, 0);
    rep(i, 0, n) pre[i + 1] = pre[i] + a[i];
    vector<vector<ll>> dp(n, vector<ll>(n, 0));  // dp[i][i] = 0
    for (int len = 2; len <= n; len++)  // short pieces first
        for (int l = 0, r = len - 1; r < n; l++, r++) {
            dp[l][r] = INF;
            rep(m, l, r)  // the last merge joins l..m and m+1..r
                dp[l][r] = min(dp[l][r], dp[l][m] + dp[m + 1][r]);
            dp[l][r] += pre[r + 1] - pre[l];  // its cost
        }
    return dp[0][n - 1];
}
```

6.15 Bitmask DP $O(2^n \cdot n^2)$

$n \leq 20$ and you must remember which ones are used: visit all (TSP, $n \leq 18$), jobs to workers ($O(2^n \cdot n)$). Narrow grid: row by row, the state is a mask of the row.

```
// shortest walk that starts at 0 and visits all n nodes (n <= 18)
// d[u][v] = distance u -> v. dp[mask][v] = shortest walk from 0
// that visited exactly the nodes in mask and ends at v
ll tsp(const vector<vector<ll>>& d) {
    int n = d.size();
    vector<vector<ll>> dp(1 << n, vector<ll>(n, INF));
    dp[1][0] = 0;  // only node 0 visited, standing at 0
    rep(mask, 1, 1 << n) rep(v, 0, n) {
        if (dp[mask][v] == INF) continue;
        rep(u, 0, n) if (!(mask >> u & 1)) {  // u not visited yet
            int nm = mask | 1 << u;
            dp[nm][u] = min(dp[nm][u], dp[mask][v] + d[v][u]);
        }
    }
    ll best = INF;
    rep(v, 0, n) best = min(best, dp[(1 << n) - 1][v]);
    return best;  // round trip: dp[(1 << n) - 1][v] + d[v][0]
}
```

```
// n workers, n jobs, c[i][j] = cost of worker i doing job j.
// dp[mask] = cheapest way to give the jobs in mask to the first
// popcount(mask) workers. O(2^n * n)
ll assign(const vector<vector<ll>>& c) {
    int n = c.size();
    vector<ll> dp(1 << n, INF);
    dp[0] = 0;
    rep(mask, 0, 1 << n) {
        int i = __builtin_popcount(mask);  // the next worker
        rep(j, 0, n) if (!(mask >> j & 1)) {  // job j is still free
            int nm = mask | 1 << j;
            dp[nm] = min(dp[nm], dp[mask] + c[i][j]);
        }
    }
    return dp[(1 << n) - 1];
}
```

6.16 Digit DP $O(digits \cdot states \cdot 10)$

Count x in $[A, B]$ with a digit property: `countUpTo(B) - countUpTo(A - 1)`.

```
// how many x in [0, N] have a digit sum divisible by K (K <= 100)?
// go(pos, rem, tight): the digits before pos are chosen, rem =
// their sum mod K, tight = they equal N's first digits
string D;  // the digits of N
int K;
ll memo[20][100][2];
ll go(int pos, int rem, bool tight) {
    if (pos == (int)D.size()) return rem == 0;
    ll& res = memo[pos][rem][tight];
    if (res != -1) return res;
    res = 0;
    int hi = tight ? D[pos] - '0' : 9;  // tight: up to N's digit
    rep(d, 0, hi + 1)
        res += go(pos + 1, (rem + d) % K, tight && d == hi);
    return res;
}
// count in [A, B]: countUpTo(B) - countUpTo(A - 1)
ll countUpTo(ll N) {
    if (N < 0) return 0;
    D = to_string(N);
    memset(memo, -1, sizeof memo);
    return go(0, 0, true);
}
```

7 Graphs

7.1 Which shortest path?

the graph	use	time
every edge costs 1	BFS	$O(n+m)$
grid, every step costs 1	grid BFS	$O(R \cdot C)$
costs only 0 or 1	0-1 BFS	$O(n+m)$
all costs ≥ 0	Dijkstra	$O(m \log n)$
some costs < 0	Bellman–Ford	$O(n \cdot m)$
all pairs, $n \leq 500$	Floyd–Warshall	$O(n^3)$
no cycles (DAG)	DP in topological order	$O(n+m)$

- Many start points: push all of them with distance 0 (multi-source).
- From every node to one target: reverse the edges, run from the target.
- The path itself: keep `par[v]`, walk back from the end.
- More than the position matters (direction, keys, turns, time mod k): the node is the state, e.g. `dist[r][c][dir]`.

7.2 Reading a graph

```
using Graph = vector<vector<pair<int, ll>>>;  // adj[u] = {to, w}
using Edges = vector<tuple<ll, int, int>>;    // list of {w, u, v}
```

```
int n, m;
cin >> n >> m;
Graph adj(n);
rep(i, 0, m) {
    int u, v; ll w;
    cin >> u >> v >> w;
    u--, v--;                    // input is 1..n, we use 0..n-1
    adj[u].push_back({v, w});
    adj[v].push_back({u, w});  // delete if edges are one-way
}
// unweighted: vector<vi> adj(n);  adj[u].push_back(v);
```

7.3 BFS $O(n+m)$

Fewest steps when every edge costs the same.

```
// dist[v] = fewest edges from s to v, -1 if unreachable
vi bfs(const vector<vi>& adj, int s) {
    vi dist(adj.size(), -1);
    queue<int> q;
    dist[s] = 0; q.push(s);  // multi-source: same for each source
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}
```

7.4 Grid BFS $O(R \cdot C)$

Maze or map of cells. 8 directions: add the diagonals to dr/dc. Count islands: start a BFS from every free cell not seen yet.

```
// '#' = wall. dist[r][c] = steps from (sr, sc), -1 = unreachable
// directions: down, up, right, left
int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
vector<vi> gridBfs(const vector<string>& g, int sr, int sc) {
    int R = g.size(), C = g[0].size();
    vector<vi> dist(R, vi(C, -1));
    queue<pii> q;
    dist[sr][sc] = 0; q.push({sr, sc});
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        rep(d, 0, 4) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            if (g[nr][nc] == '#' || dist[nr][nc] != -1) continue;
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
    return dist;
}
```

7.5 0-1 BFS $O(n + m)$

Every cost is 0 or 1 (e.g. walking is free, breaking a wall costs 1).

```
// all weights are 0 or 1. dist[v] = INF if unreachable
vector<ll> bfs01(const Graph& adj, int s) {
    vector<ll> dist(adj.size(), INF);
    deque<int> dq;
    dist[s] = 0; dq.push_back(s);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : adj[u]) if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v);  // free edge: v first
            else dq.push_back(v);
        }
    }
    return dist;
}
```

7.6 Dijkstra $O(m \log n)$

Weighted edges, all weights ≥ 0.

```
// all weights >= 0. dist[v] = INF if unreachable.
// par[v] = node before v on a shortest path (-1: none)
vector<ll> dijkstra(const Graph& adj, int s, vi& par) {
    int n = adj.size();
    vector<ll> dist(n, INF);
    par.assign(n, -1);
    // min-heap of {distance, node}: smallest distance on top
    priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                   greater<>> pq;
    dist[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;  // outdated entry: skip it
        for (auto [v, w] : adj[u]) if (d + w < dist[v]) {
            dist[v] = d + w;
            par[v] = u;
            pq.push({dist[v], v});
        }
    }
    return dist;
}
// path s -> t (check dist[t] < INF first):
//   vi path;
//   for (int v = t; v != -1; v = par[v]) path.push_back(v);
//   reverse(all(path));
```

7.7 Bellman–Ford $O(n \cdot m)$

Negative weights, or detect a negative cycle.

```
// edges {w, u, v}: u -> v with weight w (w may be negative).
// returns false if a negative cycle is reachable from s
bool bellmanFord(int n, const Edges& edges, int s,
                 vector<ll>& dist) {
    dist.assign(n, INF);
    dist[s] = 0;
    rep(round, 0, n) {
        bool changed = false;
        for (auto [w, u, v] : edges)
            if (dist[u] < INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                changed = true;
            }
        if (!changed) return true;
    }
    return false;  // still improving after n rounds
}
```

7.8 Floyd–Warshall $O(n^3)$

Distance between every pair, $n \leq 500$. Negative edges are fine.

```
// in:  d[i][j] = weight of edge i -> j, INF if there is none,
//      d[i][i] = 0
// out: d[i][j] = shortest distance from i to j.
//      d[i][i] < 0 means i is on a negative cycle
void floyd(vector<vector<ll>>& d) {
    int n = d.size();
    rep(k, 0, n)  // k = the node in the middle: OUTER loop!
        rep(i, 0, n) rep(j, 0, n)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

7.9 DFS and connected components $O(n + m)$

Which nodes are connected, how many groups there are.

```
// connected components (undirected): comp[v] = id of v's group.
// Uses the GLOBAL adj: fill it in main, don't declare a new one.
vector<vi> adj;
vi comp;
void dfs(int u, int id) {
    comp[u] = id;
    for (int v : adj[u]) if (comp[v] == -1) dfs(v, id);
}
// usage: comp.assign(n, -1); int k = 0;  // k = #components
//        rep(u, 0, n) if (comp[u] == -1) dfs(u, k++);
```

7.10 Topological sort $O(n + m)$

"a before b" rules, cycle check, DP on a DAG: for u in `order`, for $u \to v$: `best[v] = max(best[v], best[u] + 1)`.

```
// directed graph. Every edge u -> v has u before v in the result.
// result.size() < n  <=>  the graph has a cycle
vi topoSort(const vector<vi>& adj) {
    int n = adj.size();
    vi indeg(n, 0), order;
    rep(u, 0, n) for (int v : adj[u]) indeg[v]++;
    queue<int> q;
    rep(u, 0, n) if (indeg[u] == 0) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order;
}
```

7.11 Union-Find (DSU) $\approx O(1)$

Merge groups and ask "are a and b connected?" while edges are added.

```
// Union-Find: find(x) = id of x's group, unite merges 2 groups.
// Both take almost O(1).
struct DSU {
    vi p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(all(p), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {  // false if already together
        a = find(a), b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};
// same group: d.find(a) == d.find(b)
// size of a's group: d.sz[d.find(a)]
```

7.12 Minimum spanning tree (Kruskal) $O(m \log m)$

Connect all nodes as cheaply as possible. A node may buy its own source for c_v: add a node X with edges (X, v, c_v).

```
// minimum spanning tree. Needs DSU. edges = {w, u, v}.
// returns the total weight, or -1 if the graph is not connected
ll kruskal(int n, Edges edges) {
    sort(all(edges));  // tuples sort by their first field: w
    DSU dsu(n);
    ll total = 0;
    int taken = 0;
    for (auto [w, u, v] : edges)
        if (dsu.unite(u, v)) total += w, taken++;
    return taken == n - 1 ? total : -1;
}
```

8 Trees and more graphs

8.1 Tree facts

- n nodes, $n-1$ edges. Connected with $n-1$ edges $\Rightarrow$ a tree.
- Exactly one path between two nodes: one BFS gives all distances, also weighted.
- Connected graph: $m-(n-1)$ extra edges, each closes a cycle. Forest of c trees: $n-c$ edges. Few extra edges: try all 2^k ways to handle them.
- Edge above v is used by $sz[v] \cdot (n - sz[v])$ paths. Summed over all edges: the sum of all pair distances.
- Parent list with $p_i < i$: loop $i = n, \ldots, 2$ for children before parents.
- Trees are bipartite: colour by depth mod 2.

8.2 Rooting a tree $O(n)$

Parent, depth, `order`. Backwards over `order`: children before parents (sizes, tree DP). Forwards: parents first (rerooting). No recursion.

```
// root the tree at r: par[v] = parent (-1 for r), dep[v] = depth.
// Returns the BFS order: every parent comes before its children,
// so a loop over it BACKWARDS handles children before parents.
vi rootTree(const vector<vi>& adj, int r, vi& par, vi& dep) {
    int n = adj.size();
    par.assign(n, -1);
    dep.assign(n, 0);
    vi order = {r};
    for (int i = 0; i < (int)order.size(); i++) {   // it grows
        int u = order[i];
        for (int v : adj[u]) if (v != par[u]) {
            par[v] = u;
            dep[v] = dep[u] + 1;
            order.push_back(v);
        }
    }
    return order;
}
```

```
// sz[v] = number of nodes in the subtree of v (v included).
// Needs order and par from rootTree.
vi sz(n, 1);
for (int i = n - 1; i > 0; i--) sz[par[order[i]]] += sz[order[i]];
```

8.3 DFS order $O(n)$

Enter / leave times. u is in the subtree of v iff `tin[v] <= tin[u] < tout[v]`: a subtree is a range of an array.

```
// DFS order without recursion, from root r. tin[v] = when v is
// entered. The subtree of v gets the times tin[v] .. tout[v]-1,
// so u is below v exactly when tin[v] <= tin[u] < tout[v].
void dfsOrder(const vector<vi>& adj, int r, vi& tin, vi& tout) {
    int n = adj.size(), timer = 0;
    tin.assign(n, -1);
    tout.assign(n, -1);
    vi it(n, 0), st = {r};   // it[u] = next neighbour of u to try
    tin[r] = timer++;
    while (!st.empty()) {
        int u = st.back();   // depth of u = st.size() - 1
        if (it[u] < (int)adj[u].size()) {
            int v = adj[u][it[u]++];
            if (tin[v] == -1) {   // enter v, a child of u
                tin[v] = timer++;
                st.push_back(v);
            }
        } else {   // leave u: its whole subtree is done
            tout[u] = timer;
            st.pop_back();
        }
    }
}
```

8.4 Tree DP $O(n)$

The answer for v follows from its children's answers. Fill backwards over `order`; the answer is at the root.

```
// example tree DP: choose nodes, no two neighbours both chosen,
// maximise the total weight w. take[v] / skip[v] = best total
// inside the subtree of v when v is / is not chosen.
ll maxIndependentSet(const vector<vi>& adj, const vector<ll>& w) {
    int n = adj.size();
    vi par, dep;
    vi order = rootTree(adj, 0, par, dep);
    vector<ll> take(n), skip(n);
    for (int i = n - 1; i >= 0; i--) {   // children first
        int v = order[i];
        take[v] = w[v];
        skip[v] = 0;
        for (int c : adj[v]) if (c != par[v]) {   // c = a child
            take[v] += skip[c];
            skip[v] += max(take[c], skip[c]);
        }
    }
    return max(take[0], skip[0]);
}
```

8.5 Diameter $O(n)$

Longest path: BFS from any node to the farthest a, BFS from a to the farthest b. The farthest node from any v is a or b.

```
// diameter = longest path in a tree (number of edges) with its
// two ends a and b. Needs bfs() from the graphs chapter.
int diameter(const vector<vi>& adj, int& a, int& b) {
    vi d = bfs(adj, 0);
    a = max_element(all(d)) - d.begin();   // farthest from 0
    d = bfs(adj, a);
    b = max_element(all(d)) - d.begin();   // farthest from a
    return d[b];
}
```

8.6 LCA and binary lifting $O(\log n)$ per query

Distance between a and b, the k-th node above v. Weighted: $w[a] + w[b] - 2\,w[\mathrm{lca}]$. Every node has one next node (functional graph): where after $k \le 10^{18}$ steps? `Jump`.

```
// LCA = lowest common ancestor (deepest node above both a and b).
// up[j][v] = ancestor 2^j steps above v (the root above the root).
// Needs rootTree. Setup O(n log n), every query O(log n).
struct LCA {
    int LOG = 1;
    vector<vi> up;
    vi par, dep;
    LCA(const vector<vi>& adj, int root) {
        int n = adj.size();
        while ((1 << LOG) < n) LOG++;
        rootTree(adj, root, par, dep);
        up.assign(LOG, vi(n));
        rep(v, 0, n) up[0][v] = par[v] == -1 ? root : par[v];
        rep(j, 1, LOG) rep(v, 0, n)
            up[j][v] = up[j - 1][up[j - 1][v]];
    }
    int kth(int v, int k) {   // k steps up from v, -1 if too far
        if (k > dep[v]) return -1;
        rep(j, 0, LOG) if (k >> j & 1) v = up[j][v];
        return v;
    }
    int lca(int a, int b) {
        if (dep[a] < dep[b]) swap(a, b);
        a = kth(a, dep[a] - dep[b]);   // now both equally deep
        if (a == b) return a;
        // big jumps first; jump only if we stay below the LCA
        for (int j = LOG - 1; j >= 0; j--)
            if (up[j][a] != up[j][b]) a = up[j][a], b = up[j][b];
        return up[0][a];
    }
    int dist(int a, int b) {   // number of edges between a and b
        return dep[a] + dep[b] - 2 * dep[lca(a, b)];
    }
};
```

```
// every node v has one next node nxt[v]. jump(v, k) = where you
// are after k steps from v, for k up to 1e18. Memory: 60*n ints.
struct Jump {
    vector<vi> up;   // up[j][v] = node 2^j steps after v
    Jump(const vi& nxt) : up(60, nxt) {
        rep(j, 1, 60) rep(v, 0, (int)nxt.size())
            up[j][v] = up[j - 1][up[j - 1][v]];
    }
    int jump(int v, ll k) {
        rep(j, 0, 60) if (k >> j & 1) v = up[j][v];
        return v;
    }
};
```

8.7 Rerooting $O(n)$

A tree DP answer for every node as the root. Moving the root from `par[v]` to v: $sz[v]$ nodes get 1 closer, the other $n - sz[v]$ 1 farther.

```
// ans[v] = sum of the distances from v to all other nodes, for
// every v, in O(n). Needs rootTree.
vector<ll> sumOfDistances(const vector<vi>& adj) {
    int n = adj.size();
    vi par, dep;
    vi order = rootTree(adj, 0, par, dep);
    vector<ll> sz(n, 1), ans(n, 0);
    for (int i = n - 1; i > 0; i--)   // subtree sizes
        sz[par[order[i]]] += sz[order[i]];
    rep(v, 0, n) ans[0] += dep[v];   // the answer for the root
    rep(i, 1, n) {   // parents first: move the root down to v
        int v = order[i];
        ans[v] = ans[par[v]] - sz[v] + (n - sz[v]);
    }
    return ans;
}
```

8.8 Strongly connected components $O(n+m)$

Directed graph, nodes that reach each other become one node: the condensation is a DAG. To reach everything you need a start in every component with in-degree 0.

```
// strongly connected components (Kosaraju) of a directed graph.
// scc[v] = component of v, k = number of components. Ids are in
// topological order: every edge u -> v has scc[u] <= scc[v].
struct SCC {
    int n, k = 0;
    vector<vi> g, rg;   // the edges, and the edges reversed
    vi order, scc;
    vector<bool> seen;
    SCC(const vector<vi>& adj)
        : n(adj.size()), g(adj), rg(n), scc(n, -1), seen(n) {
        rep(u, 0, n) for (int v : g[u]) rg[v].push_back(u);
        rep(u, 0, n) if (!seen[u]) dfs1(u);
        reverse(all(order));   // latest finished first
        for (int u : order) if (scc[u] == -1) dfs2(u, k++);
    }
    void dfs1(int u) {   // normal DFS, remember finishing order
        seen[u] = true;
        for (int v : g[u]) if (!seen[v]) dfs1(v);
        order.push_back(u);
    }
    void dfs2(int u, int id) {   // DFS on the reversed edges
        scc[u] = id;
        for (int v : rg[u]) if (scc[v] == -1) dfs2(v, id);
    }
    vector<vi> condense() {   // the DAG with one node per component
        vector<vi> dag(k);    // (an edge may appear more than once)
        rep(u, 0, n) for (int v : g[u])
            if (scc[u] != scc[v]) dag[scc[u]].push_back(scc[v]);
        return dag;
    }
};
```

8.9 Euler path $O(n+m)$

Every edge exactly once. Undirected: 0 or 2 odd-degree nodes (start at one). Directed: in $=$ out everywhere, or one start (out $=$ in $+1$) and one end. Circuit: all even / in $=$ out.

```
// Euler path: a walk that uses every edge exactly once. Returns
// its m + 1 nodes, or {} if there is none. Edges {u, v} may
// repeat; set directed = true for one-way edges.
vi eulerPath(int n, const vector<pii>& edges, bool directed) {
    int m = edges.size();
    vector<vector<pii>> adj(n);   // adj[u] = {to, edge id}
    vi deg(n, 0);   // undirected: degree. directed: out - in
    rep(i, 0, m) {
        auto [u, v] = edges[i];
        adj[u].push_back({v, i});
        if (directed) deg[u]++, deg[v]--;
        else adj[v].push_back({u, i}), deg[u]++, deg[v]++;
    }
    int start = m ? edges[0].first : 0, odd = 0;
    rep(v, 0, n) {
        if (directed && abs(deg[v]) > 1) return {};
        if (directed ? deg[v] != 0 : deg[v] % 2 != 0) odd++;
        if (directed ? deg[v] == 1 : deg[v] % 2 != 0) start = v;
    }
    if (odd > 2) return {};
    vector<bool> used(m, false);
    vi it(n, 0), st = {start}, path;
    while (!st.empty()) {   // Hierholzer: walk until stuck,
        int u = st.back();  // then back up and add detours
        while (it[u] < (int)adj[u].size()
               && used[adj[u][it[u]].second]) it[u]++;
        if (it[u] == (int)adj[u].size()) {
            path.push_back(u);
            st.pop_back();
        } else {
            auto [v, id] = adj[u][it[u]];
            used[id] = true;
            st.push_back(v);
        }
    }
    if ((int)path.size() != m + 1) return {};   // not connected
    reverse(all(path));
    return path;
}
```

8.10 Bipartite check $O(n+m)$

Two groups, every edge between them: possible iff no odd cycle.

```
// 2-colouring: col[v] = 0 or 1, the two ends of every edge differ.
// Returns false if that is impossible (there is an odd cycle).
bool bipartite(const vector<vi>& adj, vi& col) {
    int n = adj.size();
    col.assign(n, -1);
    rep(s, 0, n) if (col[s] == -1) {   // BFS in every component
        queue<int> q;
        col[s] = 0; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (col[v] == -1) col[v] = 1 - col[u], q.push(v);
                else if (col[v] == col[u]) return false;
            }
        }
    }
    return true;
}
```

8.11 Bipartite matching $O(n \cdot m)$

Most pairs between two kinds of things. König: min vertex cover $=$ max matching; max independent set $= n -$ max matching.

```
// maximum bipartite matching (Kuhn). Left nodes 0..nl-1, right
// nodes 0..nr-1, adj[l] = right nodes that l may be paired with.
// After solve(): mt[r] = partner of r (-1 = none). O(nl * m).
struct Matching {
    vector<vi> adj;
    vi mt;
    vector<bool> seen;
    Matching(const vector<vi>& a, int nr) : adj(a), mt(nr, -1) {}
    bool augment(int l) {   // find a partner for l, maybe by
        for (int r : adj[l]) if (!seen[r]) {   // moving others
            seen[r] = true;
            if (mt[r] == -1 || augment(mt[r])) {
                mt[r] = l;
                return true;
            }
        }
        return false;
    }
    int solve() {   // returns the number of pairs
        int pairs = 0;
        rep(l, 0, (int)adj.size()) {
            seen.assign(mt.size(), false);
            if (augment(l)) pairs++;
        }
        return pairs;
    }
};
```

9 Math and number theory

9.1 Integer pitfalls

you want	not this	but this
$\lceil a/b \rceil$ for $a, b > 0$	`ceil(a/b)`	`(a+b-1)/b`
$\lfloor a/b \rfloor$, a may be < 0	`a/b`	`floorDiv(a,b)`
remainder in $[0, m)$	`a%m`	`md(a,m)`
$a \cdot b$ for two `int`s	`a*b`	`1LL*a*b`
a product up to 10^{36}	`ll`	`(__int128)a*b`
$\lfloor \sqrt{n} \rfloor$	`(ll)sqrt(n)`	`isqrt(n)`
10^{18} or 3^k	`pow(10,18)`	a loop, `modpow`
$a/b < c/d$ $(b, d > 0)$	doubles	`a*d < c*b`
round a double	`(ll)x`	`llround(x)`

```
// floor and ceil of a / b for all signs (b != 0). Plain a / b
// rounds towards 0: -7 / 2 == -3, and -7 % 3 == -1
ll floorDiv(ll a, ll b) {
    ll q = a / b;
    if (a % b != 0 && (a < 0) != (b < 0)) q--;
    return q;
}
ll ceilDiv(ll a, ll b) { return -floorDiv(-a, b); }
// a mod m in [0, m), also when a < 0: md(-7, 3) == 2
ll md(ll a, ll m) { return (a % m + m) % m; }
// exact floor(sqrt(n)) for 0 <= n <= 1e18 (sqrt alone can be 1 off)
ll isqrt(ll n) {
    ll r = sqrtl(n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    return r;
}
```

9.2 gcd, lcm and fractions $O(\log n)$

Two repeating things line up every $\operatorname{lcm}(a, b)$. All a_i equal mod m iff $m \mid \gcd(a_2 - a_1, a_3 - a_1, \dots)$. Steps of k on a circle of n visit $n/\gcd(n, k)$ spots.

```
// lcm of all numbers in a (all >= 1), or -1 as soon as it passes
// LIM. Divide before you multiply: lcm(x, y) = x / gcd(x, y) * y
ll lcmAll(const vector<ll>& a, ll LIM = 2e18) {
    ll l = 1;
    for (ll x : a) {
        ll y = x / gcd(l, x);        // lcm(l, x) = l * y
        if (l > LIM / y) return -1;  // l * y > LIM
        l *= y;
    }
    return l;
}
```

```
// exact fraction p / q, always in lowest terms with q > 0.
// Products like p * o.q must fit in ll. Print: f.p << "/" << f.q
struct Frac {
    ll p, q;
    Frac(ll a = 0, ll b = 1) {
        ll g = gcd(a, b);
        if (b < 0) g = -g;
        p = a / g, q = b / g;
    }
    Frac operator+(Frac o) { return {p * o.q + o.p * q, q * o.q}; }
    Frac operator-(Frac o) { return {p * o.q - o.p * q, q * o.q}; }
    Frac operator*(Frac o) { return {p * o.p, q * o.q}; }
    Frac operator/(Frac o) { return {p * o.q, q * o.p}; }
    bool operator<(Frac o) const { return p * o.q < o.p * q; }
    bool operator==(Frac o) const { return p == o.p && q == o.q; }
};
```

9.3 Extended Euclid: $ax + by = c$ $O(\log n)$

Solvable iff $\gcd(a, b) \mid c$. $ax \equiv c \pmod m$ is $ax + my = c$; with $c = 1$ it gives $1/a \bmod m$ for any m. Smallest $x \geq 0$ has the largest y: if that $y < 0$ there is no solution with $x, y \geq 0$.

```
// returns g = gcd(a, b) and sets x, y so that a * x + b * y = g
ll egcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1, y = 0; return a; }
    ll g = egcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
// a * x + b * y = c with a, b > 0. Returns false if no solution.
// Gives the smallest x >= 0, and its y. All solutions:
// x + k * (b / g), y - k * (a / g) for every integer k
bool solveLinear(ll a, ll b, ll c, ll& x, ll& y) {
    ll x0, y0, g = egcd(a, b, x0, y0);
    if (c % g != 0) return false;
    ll m = b / g;
    x = ((__int128)x0 * (c / g) % m + m) % m;
    y = (c - (__int128)a * x) / b;
    return true;
}
```

9.4 Modular arithmetic $O(\log e)$

`%` `MOD` after every `+` and `*`; subtract as `(a - b + MOD) % MOD`; divide by multiplying with `inv(b)` (prime `MOD`). Never compare values after `%`. $a^e \equiv a^{e \bmod (p-1)} \pmod p$.

```
const ll MOD = 1e9 + 7;  // a prime; 998244353 is another one
// b^e mod m in O(log e). Needs m < 3e9, so that m * m fits in ll
ll modpow(ll b, ll e, ll m = MOD) {
    ll r = 1;
    b = (b % m + m) % m;
    while (e > 0) {
        if (e & 1) r = r * b % m;
        b = b * b % m;
        e >>= 1;
    }
    return r;
}
// 1 / a mod p, for p prime and a % p != 0 (Fermat: a^(p-2))
ll inv(ll a, ll p = MOD) { return modpow(a, p - 2, p); }
```

9.5 Formulas and numbers

$1 + 2 + \cdots + n$	$n(n+1)/2$
$1^2 + 2^2 + \cdots + n^2$	$n(n+1)(2n+1)/6$
$1^3 + 2^3 + \cdots + n^3$	$\big(n(n+1)/2\big)^2$
$a + (a+d) + \ldots$, k terms	$k \cdot (\text{first} + \text{last})/2$
$1 + r + r^2 + \cdots + r^{n-1}$	$(r^n - 1)/(r - 1)$
$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$	$\approx \ln n + 0.58$
multiples of k in $[L, R]$	$\lfloor R/k \rfloor - \lfloor (L-1)/k \rfloor$
primes $\leq n$	25 (10^2), 168 (10^3), 78 498 (10^6), 664 579 (10^7)
most divisors	240 (10^6), 1 344 (10^9), 6 720 (10^{12}), 103 680 (10^{18})
distinct primes	at most 9 ($n \leq 10^9$), 15 ($n \leq 10^{18}$)
prime moduli	$10^9 + 7$, $10^9 + 9$, $998\,244\,353$

9.6 Sieve of Eratosthenes $O(N \log \log N)$

All primes up to N, or factorise many numbers $\leq 10^7$ (0.1 s, 40 MB).

```
// sieve: spf[x] = smallest prime factor of x (2 <= x <= MAXV),
// primes = all primes <= MAXV. x is prime iff spf[x] == x
const int MAXV = 1e7;
vi spf(MAXV + 1, 0), primes;
void sieve() {
    for (int i = 2; i <= MAXV; i++) if (spf[i] == 0) {
        primes.push_back(i);
        for (int j = i; j <= MAXV; j += i)
            if (spf[j] == 0) spf[j] = i;
    }
}
// prime factors of x (1 <= x <= MAXV) after sieve(), small first,
// with repeats: 12 gives {2, 2, 3}. O(log x)
vi factorSmall(int x) {
    vi f;
    while (x > 1) { f.push_back(spf[x]); x /= spf[x]; }
    return f;
}
```

9.7 Factorising one number $O(\sqrt{n})$

$n \leq 10^{12}$. From $n = \prod p_i^{e_i}$: divisors $\prod(e_i + 1)$, divisor sum $\prod(p_i^{e_i+1} - 1)/(p_i - 1)$, $\varphi(n) = n \prod(1 - 1/p_i)$, squarefree iff all $e_i = 1$.

```
// trial division up to sqrt(n): 1e6 steps for n = 1e12
bool isPrime(ll n) {
    if (n < 2) return false;
    for (ll d = 2; d * d <= n; d++) if (n % d == 0) return false;
    return true;
}
// prime factorisation as {prime, exponent}, primes increasing:
// 360 = 2^3 * 3^2 * 5 gives {{2, 3}, {3, 2}, {5, 1}}. n = 1: {}
vector<pair<ll, int>> factor(ll n) {
    vector<pair<ll, int>> f;
    for (ll p = 2; p * p <= n; p++) if (n % p == 0) {
        int e = 0;
        while (n % p == 0) n /= p, e++;
        f.push_back({p, e});
    }
    if (n > 1) f.push_back({n, 1});  // the rest is one big prime
    return f;
}
```

9.8 Divisors $O(\sqrt{n})$

Try every divisor of n (group sizes, periods). For all $x \leq N$ at once: loop over the multiples of every d, $O(N \log N)$.

```
// all divisors of n >= 1, sorted. O(sqrt(n)): they come in pairs
// d * (n / d) = n with d <= sqrt(n)
vector<ll> divisors(ll n) {
    vector<ll> small, big;
    for (ll d = 1; d * d <= n; d++) if (n % d == 0) {
        small.push_back(d);
        if (d != n / d) big.push_back(n / d);
    }
    small.insert(small.end(), big.rbegin(), big.rend());
    return small;
}
// cnt[x] = number of divisors of x, for every x <= N. Visits all
// multiples of every d: N/1 + N/2 + ... + N/N = O(N log N) steps
vi divisorCounts(int N) {
    vi cnt(N + 1, 0);
    for (int d = 1; d <= N; d++)
        for (int x = d; x <= N; x += d) cnt[x]++;
    return cnt;
}
```

9.9 Chinese remainder theorem $O(\log m)$

$x \equiv a_1 \pmod{m_1}$, $x \equiv a_2 \pmod{m_2}$. Solutions repeat every $\operatorname{lcm}(m_1, m_2)$; none if the moduli share a factor that the remainders disagree on.

```
// x = a1 (mod m1) and x = a2 (mod m2); m1, m2 need not be coprime.
// Returns {x, lcm(m1, m2)} with 0 <= x < lcm, or {-1, -1} if there
// is no such x. Needs egcd. The lcm must fit in ll
pair<ll, ll> crt(ll a1, ll m1, ll a2, ll m2) {
    ll p, q, g = egcd(m1, m2, p, q);  // m1 * p + m2 * q = g
    if ((a2 - a1) % g != 0) return {-1, -1};
    ll l = m1 / g * m2;
    // x = a1 + m1 * k, where m1 * k = a2 - a1 (mod m2)
    ll k = (__int128)((a2 - a1) / g) * p % (m2 / g);
    ll x = ((a1 + (__int128)m1 * k) % l + l) % l;
    return {x, l};
}
// more equations: merge one at a time, {x, m} = crt(x, m, a3, m3)
```

9.10 Permutation cycles $O(n)$

Repeated shuffle: after k steps every element moves $k \bmod L$ along its cycle of length L. Back to start after the lcm of the lengths. Sorting with swaps: $n - \#\text{cycles}$.

```
// cycles of a permutation p of 0..n-1, where i goes to p[i]
vector<vi> cycles(const vi& p) {
    int n = p.size();
    vector<bool> seen(n, false);
    vector<vi> cyc;
    rep(i, 0, n) if (!seen[i]) {
        cyc.push_back({});
        for (int j = i; !seen[j]; j = p[j]) {
            seen[j] = true;
            cyc.back().push_back(j);
        }
    }
    return cyc;
}
// p applied k times (k up to 1e18): q[i] = p[p[...p[i]...]].
// Along its cycle, i moves k % len places. O(n)
vi applyK(const vi& p, ll k) {
    vi q(p.size());
    for (auto& c : cycles(p)) {
        int len = c.size();
        rep(j, 0, len) q[c[j]] = c[(j + k) % len];
    }
    return q;
}
```

9.11 Matrix power $O(k^3 \log n)$

$f(n) = 2f(n-1) + 3f(n-2)$ for $n \le 10^{18}$: first row the coefficients, below it a shifted identity; $f(n) = R[0][0]f(1) + R[0][1]f(0)$ for $R = M^{n-1}$. Walks of length L: A^L.

```
// matrix product and power mod MOD (needs MOD). For k x k matrices
// matpow takes O(k^3 log e)
using Mat = vector<vector<ll>>;
Mat matmul(const Mat& A, const Mat& B) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Mat C(n, vector<ll>(m, 0));
    rep(i, 0, n) rep(j, 0, m) rep(l, 0, k)
        C[i][j] = (C[i][j] + A[i][l] * B[l][j]) % MOD;
    return C;
}
Mat matpow(Mat A, ll e) {
    int n = A.size();
    Mat R(n, vector<ll>(n, 0));
    rep(i, 0, n) R[i][i] = 1;    // identity
    while (e > 0) {
        if (e & 1) R = matmul(R, A);
        A = matmul(A, A);
        e >>= 1;
    }
    return R;
}
// Fibonacci: F(n) = matpow({{1, 1}, {1, 0}}, n)[0][1]
```

9.12 XOR tricks $O(n \log V)$

$x \oplus x = 0$; range xor via prefix xors; solve each bit on its own. Tree path $u \to v$: $d[u] \oplus d[v]$. Graph walks: insert $d[a] \oplus d[b] \oplus w$ for every non-tree edge into the basis.

```
// xor basis: which values are the xor of some subset of the
// numbers inserted so far (numbers < 2^61). O(61) per operation
struct XorBasis {
    ll b[61] = {};  // b[i]: a number with highest bit i, or 0
    bool insert(ll x) {  // false: x was already a subset xor
        for (int i = 60; i >= 0; i--) if (x >> i & 1) {
            if (b[i] == 0) { b[i] = x; return true; }
            x ^= b[i];
        }
        return false;
    }
    // smallest / largest value of x ^ (xor of some subset)
    ll minXor(ll x) {
        for (int i = 60; i >= 0; i--) x = min(x, x ^ b[i]);
        return x;
    }
    ll maxXor(ll x) {
        for (int i = 60; i >= 0; i--) x = max(x, x ^ b[i]);
        return x;
    }
};
```

9.13 Big numbers $O(digits)$

Hundreds of digits. Or submit that problem in Python 3: its integers never overflow.

```
// non-negative big numbers as decimal strings without leading
// zeros, e.g. "12345". All O(length), multiply O(|a| * |b|)
bool lessBig(const string& a, const string& b) {   // a < b
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}
string addBig(const string& a, const string& b) {
    string r;
    int i = a.size() - 1, j = b.size() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        if (i >= 0) carry += a[i--] - '0';
        if (j >= 0) carry += b[j--] - '0';
        r += char('0' + carry % 10);
        carry /= 10;
    }
    reverse(all(r));
    return r;
}
string subBig(const string& a, const string& b) {   // needs a >= b
    string r;
    int i = a.size() - 1, j = b.size() - 1, borrow = 0;
    while (i >= 0) {
        int s = a[i--] - '0' - borrow - (j >= 0 ? b[j--] - '0' : 0);
        borrow = s < 0;
        r += char('0' + (s + 10) % 10);
    }
    while (r.size() > 1 && r.back() == '0') r.pop_back();
    reverse(all(r));
    return r;
}
string mulBig(const string& a, const string& b) {
    vi d(a.size() + b.size(), 0);
    for (int i = a.size() - 1; i >= 0; i--)
        for (int j = b.size() - 1; j >= 0; j--)
            d[i + j + 1] += (a[i] - '0') * (b[j] - '0');
    for (int k = d.size() - 1; k > 0; k--) {
        d[k - 1] += d[k] / 10;
        d[k] %= 10;
    }
    string r;
    for (int x : d) if (!r.empty() || x) r += char('0' + x);
    return r.empty() ? "0" : r;
}
// a / m and a % m for a small m (m up to 1e17)
string divSmall(const string& a, ll m, ll& rem) {
    string q;
    rem = 0;
    for (char c : a) {
        rem = rem * 10 + (c - '0');
        q += char('0' + rem / m);
        rem %= m;
    }
    size_t p = q.find_first_not_of('0');
    return p == string::npos ? "0" : q.substr(p);
}
```

```python
# Python 3: ints never overflow, Fraction is exact
import sys
from fractions import Fraction
a, b = map(int, sys.stdin.readline().split())
print(a * b, a // b, a % b)   # // and % round down, also < 0
print(pow(a, b, 10**9 + 7))   # a^b mod m, fast
print(Fraction(a, b) + Fraction(1, 6))   # exact, prints p/q
print(str(a ** 50)[:10], len(str(a ** 50)))   # huge numbers
```

10 Counting and probability

10.1 Counting formulas

what you count	how many
orders of n different things	$n!$
pick k of n, order matters	$n!/(n-k)!$
pick k of n, order does not matter	$C(n,k)$
subsets of n things	2^n
strings of length k over n letters	n^k
orders of a word, letter i appears c_i times	$n!/(c_1!\cdots c_m!)$
grid paths, a right and b up	$C(a+b,a)$
$x_1+\cdots+x_k=s$, all $x_i\geq 0$	$C(s+k-1,k-1)$
$x_1+\cdots+x_k=s$, all $x_i\geq 1$	$C(s-1,k-1)$
pick k of n kinds, repeats allowed	$C(n+k-1,k)$
n people at a round table	$(n-1)!$
derangements D_n (nobody in place)	$(n-1)(D_{n-1}+D_{n-2})$
Catalan: brackets, binary trees	$C(2n,n)/(n+1)$

D_n: 1, 0, 1, 2, 9, 44, 265. **Catalan:** 1, 1, 2, 5, 14, 42, 132, 429, 1430.

10.2 Binomial coefficients mod p $O(1)$ per query

$C(n,k)$ or $n!$ modulo a prime, $n \leq 10^6$.

```cpp
// nCr(n, k) mod MOD for 0 <= n < MAXN, O(1) after initFact().
// Needs MOD (a prime >= MAXN) and modpow from the Math chapter
const int MAXN = 1e6 + 1;
ll fact[MAXN], ifact[MAXN];  // n! and 1 / n!
void initFact() {
    fact[0] = 1;
    rep(i, 1, MAXN) fact[i] = fact[i - 1] * i % MOD;
    ifact[MAXN - 1] = modpow(fact[MAXN - 1], MOD - 2);
    for (int i = MAXN - 1; i > 0; i--)
        ifact[i - 1] = ifact[i] * i % MOD;  // 1/(i-1)! = i * 1/i!
}
ll nCr(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
}
```

10.3 Pascal's triangle $O(n^2)$

Small n, a modulus that is not prime, or $C(n,k)$ as a `double`.

```cpp
// C[n][k] for n, k < M by Pascal's rule. Exact in ll up to n = 66;
// for any modulus (also not prime) add % m; use double for
// probabilities with n up to about 1000
int M = 67;
vector<vector<ll>> C(M, vector<ll>(M, 0));
rep(n, 0, M) {
    C[n][0] = 1;
    rep(k, 1, n + 1) C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
}
```

10.4 Inclusion–exclusion $O(2^k \cdot k)$

At least one (or none) of $k \leq 20$ properties: over every non-empty set S, $+$ the count with all of S if $|S|$ is odd, $-$ if even. Bounded stars and bars ($x_i \leq u$):
$\sum_j (-1)^j C(k,j)\, C(s-j(u+1)+k-1, k-1)$.

```cpp
// how many x in 1..n are divisible by at least one of a[0..k-1]?
// Inclusion-exclusion over all subsets S of a (k <= 20): add
// n / lcm(S) if |S| is odd, subtract it if even. n, a[i] <= 1e9
ll countDivisible(ll n, const vector<ll>& a) {
    int k = a.size();
    ll total = 0;
    rep(mask, 1, 1 << k) {
        ll l = 1;
        rep(i, 0, k) if (mask >> i & 1) {
            l = l / gcd(l, a[i]) * a[i];
            if (l > n) break;  // n / l is 0 from here on
        }
        if (__builtin_popcount(mask) % 2 == 1) total += n / l;
        else total -= n / l;
    }
    return total;
}
```

10.5 Probability rules

- At least one $= 1 - P(\text{none})$, e.g. $1-(1-p)^n$.
- Independent: $P(A \text{ and } B) = P(A)\cdot P(B)$.
  $P(A \mid B) = P(A \text{ and } B)/P(B)$.
- Success with probability p per try: on average $1/p$ tries.
- **Linearity:** $E[X+Y] = E[X] + E[Y]$, always. Expected count $= \sum_{\text{items}} P(\text{item counts})$.
- All k sides of a die seen: $k(\frac{1}{k} + \frac{1}{k-1} + \cdots + \frac{1}{1})$ throws.

10.6 Expected-value DP states $\times$ moves

$E[s] = cost(s) + \sum_t P(s \to t)E[t]$, $E = 0$ at the end. Self-loop with probability q: $E[s] = (c + rest)/(1-q)$. You choose: take the $\min$.

```cpp
// expected-value DP. Example: you stand on square i of 0..n; each
// turn throw a fair die 1..k and move forward, but a throw past n
// is wasted (you stay). E[i] = expected turns from i until n
vector<double> E(n + 1, 0);  // E[n] = 0: already there
for (int i = n - 1; i >= 0; i--) {
    double sum = 0, stay = 0;  // stay = P(this throw is wasted)
    rep(r, 1, k + 1) {
        if (i + r <= n) sum += E[i + r] / k;
        else stay += 1.0 / k;
    }
    // E[i] = 1 + sum + stay * E[i]: move stay * E[i] to the left
    E[i] = (1 + sum) / (1 - stay);
}
```

10.7 Probability DP steps $\times$ states

Probability of every outcome after random steps: `nd[t] += dp[s] * q`. The values must add up to 1.

```cpp
// probability DP: dp[j] = P(being in state j after the steps so
// far); each step pushes every state's probability forward.
// Example: n tries, each succeeds with probability p;
// at the end dp[j] = P(exactly j successes). O(n^2)
vector<double> dp(n + 1, 0);
dp[0] = 1;  // before any try: 0 successes, for sure
rep(i, 0, n) {
    vector<double> nd(n + 1, 0);  // the states after try i
    rep(j, 0, i + 1) {
        nd[j] += dp[j] * (1 - p);   // try i fails
        nd[j + 1] += dp[j] * p;     // try i succeeds
    }
    dp = nd;
}
```

10.8 Answers as a fraction mod p

"Print $P \cdot Q^{-1} \bmod p$": work mod p throughout, a/b becomes `frac(a, b)`, divide by multiplying with `inv`.

```cpp
// the fraction p / q as one number mod MOD (prime), for "print
// P * Q^-1 mod MOD". Needs MOD and inv. frac(1, 2) == (MOD + 1) / 2
ll frac(ll p, ll q) { return (p % MOD + MOD) % MOD * inv(q) % MOD; }
```

10.9 Doubles

- Print with `fixed << setprecision(10)` (in the template).
- Tiny probabilities (0.5^{2000}) underflow: add logarithms, `exp` at the end.
- Unsure about a formula? Simulate 10^6 times and compare on the samples.

11 Games

11.1 Winning and losing

- Losing: every move leads to a winning position (or there is no move). Winning: some move leads to a losing one. Fill from the end positions.
- Symmetric game: copying the other player's move often wins.
- No idea? Print `win[]` for small n and look for a pattern (often periodic).

11.2 Win / lose table $O(n \cdot |S|)$

One pile, a move takes $s \in S$ stones, who cannot move loses.

```
// a pile of i stones; a move takes s stones for some s in S; who
// cannot move loses. win[i] = the player to move wins
vector<bool> winning(int n, const vi& S) {
    vector<bool> win(n + 1, false);   // win[0] = false: no move
    rep(i, 1, n + 1) for (int s : S)
        if (s <= i && !win[i - s]) win[i] = true;  // move to a loss
    return win;
}
```

11.3 Nim and Grundy numbers $O(n \cdot |S|)$

Several independent games: xor of the Grundy numbers, 0 = you lose. Nim: xor of the pile sizes; winning move: pile p becomes $p \oplus x$ where x is the xor.

```
// Grundy number of every pile size 0..n for the move set S.
// Several independent piles: xor their Grundy numbers; the player
// to move loses iff the xor is 0. Plain Nim (take any amount):
// the Grundy number of a pile is its size
vi grundy(int n, const vi& S) {
    vi g(n + 1, 0);
    rep(i, 1, n + 1) {
        set<int> reach;   // Grundy numbers you can move to
        for (int s : S) if (s <= i) reach.insert(g[i - s]);
        while (reach.count(g[i])) g[i]++;   // mex: smallest missing
    }
    return g;
}
```

11.4 Taking from both ends $O(n^2)$

Players take the first or last element, each wants the most.

```
// players take turns taking a[l] or a[r]; each wants the most for
// themselves. d[l][r] = mover's total - other's total on a[l..r]
vector<vector<ll>> d(n, vector<ll>(n, 0));
for (int l = n - 1; l >= 0; l--) {
    d[l][l] = a[l];
    rep(r, l + 1, n)
        d[l][r] = max(a[l] - d[l + 1][r], a[r] - d[l][r - 1]);
}
// first player gets (sum of a + d[0][n - 1]) / 2
```

12 Geometry

12.1 Rules

- Integer input: `T = ll`, stay exact (cross, squared lengths, `area2`). Coordinates up to 10^9 fit. `sqrt` and division only at the end.
- Doubles: never ==; use `sgn`. `EPS = 1e-9` for coordinates up to 10^3, larger coordinates need a larger `EPS`.
- Distances: compare `norm(p - q) <= r * r`, not square roots.
- Radians: `deg * M_PI / 180`. Direction: `atan2(y, x)`, never `atan(y / x)`.
- NaN: `acos(min(1.0, max(-1.0, v)))`, `sqrt(max(0.0, v))`.
- Polygon: clockwise or not (sign of `area2`)? First point repeated at the end?

12.2 Point and cross product

Copy first. dot > 0: angle $< 90°$, $= 0$: perpendicular. `cross(a, b)` > 0: b is left of a, $= 0$: parallel.

```
// 2D point / vector. T = ll: exact integer math (when the input is
// integer). T = double: decimals in the input, or you need division
using T = ll;
const double EPS = 1e-9;
int sgn(T v) { return (v > EPS) - (v < -EPS); }  // -1, 0 or 1
struct P {
    T x, y;
    P operator+(P o) const { return {x + o.x, y + o.y}; }
    P operator-(P o) const { return {x - o.x, y - o.y}; }
    P operator*(T k) const { return {x * k, y * k}; }
    P operator/(T k) const { return {x / k, y / k}; }
    bool operator<(P o) const { return tie(x, y) < tie(o.x, o.y); }
    bool operator==(P o) const { return x == o.x && y == o.y; }
};
istream& operator>>(istream& in, P& p) { return in >> p.x >> p.y; }
T dot(P a, P b) { return a.x * b.x + a.y * b.y; }
T cross(P a, P b) { return a.x * b.y - a.y * b.x; }
T norm(P a) { return dot(a, a); }   // squared length (exact)
double len(P a) { return sqrt((double)norm(a)); }
// > 0: a -> b -> c turns left (counter-clockwise), < 0: turns
// right, 0: all on one line. |cross(a, b, c)| = 2 * triangle area
T cross(P a, P b, P c) { return cross(b - a, c - a); }
```

12.3 Segment intersection $O(1)$

Do two segments meet? Is a point on a segment? Exact with integers.

```
// is p on the segment [a, b]? (the endpoints count)
bool onSeg(P p, P a, P b) {
    return sgn(cross(a, b, p)) == 0 && sgn(dot(a - p, b - p)) <= 0;
}
// do segments [a, b] and [c, d] have a common point? Touching and
// overlapping count. Exact with T = ll
bool segsIntersect(P a, P b, P c, P d) {
    int s1 = sgn(cross(a, b, c)), s2 = sgn(cross(a, b, d));
    int s3 = sgn(cross(c, d, a)), s4 = sgn(cross(c, d, b));
    if (s1 * s2 < 0 && s3 * s4 < 0) return true;  // a proper X
    return onSeg(c, a, b) || onSeg(d, a, b) ||
           onSeg(a, c, d) || onSeg(b, c, d);
}
```

12.4 Lines and distances $O(1)$

Crossing point of two lines, foot of a perpendicular, distance to a line or segment.

```
// T = double. Point where the infinite lines a-b and c-d cross;
// false if they are parallel. Two segments: segsIntersect first
bool lineInter(P a, P b, P c, P d, P& out) {
    T den = cross(b - a, d - c);
    if (sgn(den) == 0) return false;
    out = a + (b - a) * (cross(c - a, d - c) / den);
    return true;
}
// T = double. Point on the line a-b closest to p
P project(P p, P a, P b) {
    return a + (b - a) * (dot(p - a, b - a) / norm(b - a));
}
// distance from p to the infinite line a-b (a != b)
double distLine(P p, P a, P b) {
    return abs(cross(a, b, p)) / len(b - a);
}
// distance from p to the segment [a, b]
double distSeg(P p, P a, P b) {
    if (a == b || dot(p - a, b - a) <= 0) return len(p - a);
    if (dot(p - b, a - b) <= 0) return len(p - b);
    return distLine(p, a, b);
}
// segment to segment: 0 if segsIntersect, else the minimum of the
// 4 distances from an endpoint to the other segment
```

12.5 Polygon area, point in polygon $O(n)$

Shoelace area; inside / on border / outside. In triangle abc: `cross(a,b,q)`, `cross(b,c,q)`, `cross(c,a,q)` all ≥ 0 or all ≤ 0.

```
// twice the signed area of a polygon (vertices in order). > 0 if
// counter-clockwise. For integer points this is an exact integer
T area2(const vector<P>& p) {
    int n = p.size();
    T s = 0;
    rep(i, 0, n) s += cross(p[i], p[(i + 1) % n]);
    return s;
}
// is q in the polygon (any simple one, convex or not)? O(n)
// 1 = inside, 0 = on the border, -1 = outside
int inPolygon(const vector<P>& p, P q) {
    int n = p.size();
    bool in = false;
    rep(i, 0, n) {
        P a = p[i], b = p[(i + 1) % n];
        if (onSeg(q, a, b)) return 0;
        if (a.y > b.y) swap(a, b);   // now a is the lower end
        // does this edge cross the ray from q to the right?
        if (a.y <= q.y && q.y < b.y && sgn(cross(a, b, q)) > 0)
            in = !in;
    }
    return in ? 1 : -1;
}
```

12.6 What shape is it?

Use *squared* lengths. **Triangle**, sides sorted $a \leq b \leq c$: right iff $a + b = c$, acute iff $a + b > c$, obtuse iff $a + b < c$; collinear points are no triangle. **Quadrilateral** $ABCD$, sides $s_1..s_4$ (AB, BC, CD, DA), diagonals $d_1 = AC$, $d_2 = BD$. Most specific first:

- square: 4 sides equal, $d_1 = d_2$; rhombus: 4 sides equal
- rectangle: $s_1 = s_3$, $s_2 = s_4$, $d_1 = d_2$; parallelogram: $s_1 = s_3$, $s_2 = s_4$
- kite: $(s_1 = s_2, s_3 = s_4)$ or $(s_2 = s_3, s_4 = s_1)$
- trapezium: `cross(B-A, D-C) == 0` or `cross(C-B, A-D) == 0`

12.7 Convex hull $O(n \log n)$

Smallest convex fence, its length or area. The two farthest points are hull vertices.

```
// convex hull, counter-clockwise, without points in the middle of
// an edge. Fewer than 3 points if all points are on one line
vector<P> hull(vector<P> pts) {
    sort(all(pts));
    pts.erase(unique(all(pts)), pts.end());
    if (pts.size() < 3) return pts;
    vector<P> h;
    rep(pass, 0, 2) {   // lower half, then upper half
        int start = h.size();
        for (P p : pts) {
            // drop the last point while it does not turn left
            while ((int)h.size() >= start + 2 &&
                   sgn(cross(h[h.size() - 2], h.back(), p)) <= 0)
                h.pop_back();
            h.push_back(p);
        }
        h.pop_back();   // it starts the other half
        reverse(all(pts));
    }
    return h;
}
```

12.8 Closest pair of points $O(n \log n)$

All pairs is fine up to $n \approx 10^4$. Random points: sort by x, look back only while $(\Delta x)^2 <$ best.

```
// T = ll. Smallest SQUARED distance between two of the points
// (n >= 2). Sweep left to right; s = points with x > q.x - d,
// sorted by y. O(n log n)
ll closestPair(vector<P> p) {
    sort(all(p));
    set<pair<ll, ll>> s;   // {y, x}
    ll best = LLONG_MAX;
    int j = 0;
    for (P q : p) {
        ll d = sqrtl(best) + 1;   // farther points cannot win
        while (q.x - p[j].x >= d) s.erase({p[j].y, p[j].x}), j++;
        auto it = s.lower_bound({q.y - d, LLONG_MIN});
        for (; it != s.end() && it->first <= q.y + d; it++)
            best = min(best, norm(q - P{it->second, it->first}));
        s.insert({q.y, q.x});
    }
    return best;
}
```

12.9 Angles and rotation

Direction `atan2(p.y, p.x)` in $(-\pi, \pi]$. Angle at b: `angle(a - b, c - b)`. `byAngle`: exact sort by direction.

```
// angle between vectors a and b, in [0, pi] (radians)
double angle(P a, P b) {
    return atan2(abs(cross(a, b)), dot(a, b));
}
// sort vectors by direction, counter-clockwise, starting at the
// +x axis: sort(all(v), byAngle). Exact with T = ll. No (0, 0)!
bool lowHalf(P a) { return a.y < 0 || (a.y == 0 && a.x < 0); }
bool byAngle(P a, P b) {
    if (lowHalf(a) != lowHalf(b)) return lowHalf(b);
    return sgn(cross(a, b)) > 0;
}
```

```
// T = double. Rotate p counter-clockwise by t radians around the
// origin. Around a centre c: rot(p - c, t) + c
P rot(P p, double t) {
    double c = cos(t), s = sin(t);
    return {p.x * c - p.y * s, p.x * s + p.y * c};
}
// turn 90 degrees left, exact: {-p.y, p.x}. Right: {p.y, -p.x}
```

12.10 Circles

p in circle: `norm(p - c) <= r * r`. Discs overlap iff $|c_1c_2|^2 \leq (r_1 + r_2)^2$. Connected discs: test all pairs, then DSU.

```
// T = double. Where the line a-b meets the circle (c, r): 0, 1 or
// 2 points
vector<P> lineCircle(P a, P b, P c, double r) {
    P d = b - a;
    P m = a + d * (dot(c - a, d) / norm(d));   // closest to c
    double h2 = r * r - norm(c - m);
    if (h2 < -EPS) return {};
    if (h2 < EPS) return {m};   // the line touches the circle
    P off = d * (sqrt(h2) / len(d));
    return {m - off, m + off};
}
// T = double. Where the circles (a, r1) and (b, r2) meet
vector<P> circleCircle(P a, double r1, P b, double r2) {
    P v = b - a;
    double d2 = norm(v);
    if (d2 < EPS) return {};   // same centre
    double x = (d2 + r1 * r1 - r2 * r2) / (2 * d2);  // part of v
    double h2 = r1 * r1 / d2 - x * x;
    if (h2 < -EPS) return {};
    P m = a + v * x, off = P{-v.y, v.x} * sqrt(max(0.0, h2));
    if (h2 < EPS) return {m};
    return {m - off, m + off};
}
```

12.11 Formulas

- Pick: area $= I + B/2 - 1$, so $I = (|\texttt{area2}| - B + 2)/2$, with $B = \sum_{\text{edges}} \gcd(|\Delta x|, |\Delta y|)$.
- $c^2 = a^2 + b^2 - 2ab\cos\gamma, \quad a/\sin\alpha = 2R$.
- Heron: area $= \sqrt{s(s-a)(s-b)(s-c)}$, $s = (a+b+c)/2$. $R = abc/(4A)$, $r = A/s$.
- Polygon with n corners: inside angles sum to $(n-2)\cdot 180°$.
- Circle: $2\pi r$, πr^2, arc $r\theta$, sector $r^2\theta/2$, chord $2r\sin(\theta/2)$. Sphere: $4\pi r^2$, $\frac{4}{3}\pi r^3$.

13 Strings

13.1 Which string tool?

the problem	use	time
anagrams, letter frequencies	count array	$O(n)$
letters form a palindrome?	parity mask	$O(n)$
longest palindromic substring	grow from centres	$O(n^2)$
find a pattern once	`s.find(p)`	$O(nm)$
all occurrences, period	KMP	$O(n+m)$
compare many substrings	hashing	$O(1)$ each
many words, prefix questions	trie	$O(\text{length})$
LCS, edit distance	DP chapter	$O(nm)$

$n \le 1000$: trying every substring with `substr` is often fine.

13.2 Counting letters $O(n)$

Anagrams, "can A be made from the letters of B?". Per position: `cnt[i][c]`. In a range: prefix sums per letter.

```
// cnt[c] = how often letter c occurs in s ('a' -> 0, 'z' -> 25).
// Upper case: ch - 'A'. Any character: int cnt[256] = {};
// and cnt[(unsigned char)ch]++
int cnt[26] = {};
for (char ch : s) cnt[ch - 'a']++;
// anagrams (same letters, other order): same cnt arrays
```

13.3 Palindromes $O(n^2)$

Longest palindrome inside a string: grow from each of the $2n-1$ centres.

```
bool isPal(const string& s) { return equal(all(s), s.rbegin()); }
// longest palindromic substring in O(n^2): grow around every
// centre (a letter for odd lengths, a gap for even). {start, len}
pii longestPal(const string& s) {
    int n = s.size(), start = 0, best = 0;
    rep(c, 0, 2 * n - 1) {   // c even: centre s[c/2]; odd: a gap
        int l = c / 2, r = l + c % 2;
        while (l >= 0 && r < n && s[l] == s[r]) l--, r++;
        if (r - l - 1 > best) start = l + 1, best = r - l - 1;
    }
    return {start, best};   // the text: s.substr(start, best)
}
```

13.4 KMP $O(n+m)$

All occurrences of a pattern; the period of a string. b is a rotation of a iff $|a| = |b|$ and b occurs in $a + a$.

```
// pi[i] = length of the longest prefix of s[0..i] that is also a
// suffix of s[0..i] (not the whole s[0..i] itself). O(n)
vi prefixFunction(const string& s) {
    int n = s.size();
    vi pi(n, 0);
    rep(i, 1, n) {
        int k = pi[i - 1];
        while (k > 0 && s[i] != s[k]) k = pi[k - 1];  // try shorter
        if (s[i] == s[k]) k++;
        pi[i] = k;
    }
    return pi;
}
// start of every occurrence of pat in txt, overlaps too. O(n + m)
// '#' must be a character that is in neither string
vi findAll(const string& pat, const string& txt) {
    int m = pat.size();
    vi pi = prefixFunction(pat + '#' + txt), res;
    rep(i, 0, (int)pi.size())
        if (pi[i] == m) res.push_back(i - 2 * m);
    return res;
}
// period: the smallest p with s[i] == s[i + p] for all i is
// p = n - pi[n - 1]. s is s[0..p-1] repeated iff n % p == 0
```

13.5 Rolling hash $O(1)$ per substring

Compare many substrings. Longest repeated substring: binary search on the length, hashes in a set.

```
// polynomial hash mod 2^61 - 1. After O(n) setup, get(l, r) is the
// hash of s[l..r-1] in O(1). Equal strings give equal hashes;
// different ones almost never do (chance about n / 2^61)
using ull = unsigned long long;
const ull HMOD = (1ULL << 61) - 1, HB = 1e9 + 7;  // HB: big, < HMOD
ull hmul(ull a, ull b) { return (__uint128_t)a * b % HMOD; }
struct Hash {
    vector<ull> h, pw;   // h[i] = hash of s[0..i-1], pw[i] = HB^i
    Hash(const string& s) : h(s.size() + 1), pw(s.size() + 1) {
        pw[0] = 1;
        rep(i, 0, (int)s.size()) {
            h[i + 1] = (hmul(h[i], HB) + s[i]) % HMOD;
            pw[i + 1] = hmul(pw[i], HB);
        }
    }
    ull get(int l, int r) {   // hash of s[l..r-1]
        return (h[r] + HMOD - hmul(h[l], pw[r - l])) % HMOD;
    }
};
// Hash hs(s); if (hs.get(i, i + L) == hs.get(j, j + L)) same text
// two strings: one Hash for each, then compare get() values
```

13.6 Trie $O(\textbf{length})$

Many words, "how many start with . . . ?". Without a trie: sort, the words starting with p are `lower_bound(p)` to `lower_bound(p + '{')`.

```
// trie of lowercase words. Node 0 = root = the empty prefix.
// cnt[v] = number of words that start with the prefix of node v,
// ends[v] = number of words equal to it
struct Trie {
    vector<array<int, 26>> nxt;   // nxt[v][c] = child, 0 = none
    vi cnt, ends;
    Trie() { newNode(); }
    int newNode() {
        nxt.push_back({}), cnt.push_back(0), ends.push_back(0);
        return nxt.size() - 1;
    }
    void insert(const string& w) {
        int v = 0;
        cnt[0]++;
        for (char ch : w) {
            int c = ch - 'a';
            if (nxt[v][c] == 0) {
                int u = newNode();   // first: it resizes nxt
                nxt[v][c] = u;
            }
            v = nxt[v][c];
            cnt[v]++;
        }
        ends[v]++;
    }
    int find(const string& p) {   // node of prefix p, -1 if none
        int v = 0;
        for (char ch : p) {
            v = nxt[v][ch - 'a'];
            if (v == 0) return -1;
        }
        return v;
    }
};
// words starting with p: int v = t.find(p); v < 0 ? 0 : t.cnt[v]
```

13.7 Palindrome by rearranging $O(n)$

Possible iff at most one letter has an odd count: one bit per letter, `mask ^= 1 << c`. Substring $s[l..r-1]$: `pre[r] ^ pre[l]`.

```
// bit c of the mask is 1 iff letter c occurs an odd number of
// times. s can be rearranged into a palindrome iff <= 1 bit is set
bool canBePal(const string& s) {
    int mask = 0;
    for (char ch : s) mask ^= 1 << (ch - 'a');
    return __builtin_popcount(mask) <= 1;
}
// number of substrings that can be rearranged into a palindrome.
// s[l..r-1] has mask pre[r] ^ pre[l] (pre = mask of a prefix), so
// count earlier prefixes whose mask differs in at most 1 bit
ll palSubstrings(const string& s) {
    map<int, ll> seen{{0, 1}};   // mask -> how many prefixes
    int mask = 0;
    ll res = 0;
    for (char ch : s) {
        mask ^= 1 << (ch - 'a');
        rep(c, -1, 26) {   // c = -1: all even, else only c odd
            int want = c < 0 ? mask : mask ^ (1 << c);
            if (seen.count(want)) res += seen[want];
        }
        seen[mask]++;
    }
    return res;
}
```

14 Brute force and testing

14.1 All subsets $O(2^n \cdot n)$

$n \leq 20$, choose some of the items. A few special things with 2 options each: try all 2^k.

```
// try every subset of the n items (n <= 20): bit i of mask says
// if item i is in it. Example: can some of the a[i] sum to S?
bool found = false;
rep(mask, 0, 1 << n) {
    ll sum = 0;
    rep(i, 0, n) if (mask >> i & 1) sum += a[i];
    if (sum == S) found = true;
}
// number of items in the subset: __builtin_popcount(mask)
```

14.2 All orders $O(n! \cdot n)$

$n \leq 10$ and the answer is an order (route, schedule, assignment).

```
// try every order of n items (n <= 10: 10! = 3.6e6 orders).
// Example: shortest route through all n cities, d = distances.
vi p(n);
iota(all(p), 0);   // 0, 1, ..., n-1: must start sorted!
ll best = INF;
do {
    ll len = 0;
    rep(i, 0, n - 1) len += d[p[i]][p[i + 1]];
    best = min(best, len);
} while (next_permutation(all(p)));
```

14.3 Backtracking with pruning

Too many ways, but bad partial answers can be stopped early. Hardest items first; skip options equal to one already tried.

```
// backtracking with pruning. Example: put every job on one of the
// machines so that the busiest machine has as little work as
// possible. Sort the jobs big to small first: prunes much more.
vector<ll> job, load;   // load[k] = total work on machine k
ll best = INF;          // best answer found so far
void place(int i, ll mx) {   // jobs 0..i-1 placed, mx = max load
    if (mx >= best) return;  // prune: cannot beat the best
    if (i == (int)job.size()) { best = mx; return; }
    rep(k, 0, (int)load.size()) {
        load[k] += job[i];               // choose
        place(i + 1, max(mx, load[k]));  // go deeper
        load[k] -= job[i];               // undo
        if (load[k] == 0) break;   // empty machines are all alike
    }
}
// sort(job.rbegin(), job.rend()); load.assign(machines, 0);
// best = INF; place(0, 0);  // answer in best
```

14.4 Stress test

WA and no idea why: `brute.cpp` (slow, surely right), `gen.cpp` (small random tests), `sol.cpp`; run until the outputs differ. Crash or hang: the test is in `in.txt`. Too slow? `time ./sol < big.txt` with a worst-case input.

```
// random test generator. "./gen 5 > in.txt" writes test 5: the
// same seed always gives the same test. Put the template above.
int main(int argc, char* argv[]) {
    mt19937 rng(atoi(argv[1]));
    auto rnd = [&](int lo, int hi) {   // random int in [lo, hi]
        return uniform_int_distribution<int>(lo, hi)(rng);
    };
    int n = rnd(1, 8);   // keep tests SMALL: easy to check by hand
    cout << n << '\n';
    rep(i, 0, n) cout << rnd(1, 10) << " \n"[i == n - 1];
    // tree on nodes 1..n, one line "parent child" per edge:
    // rep(v, 2, n + 1) cout << rnd(1, v - 1) << ' ' << v << '\n';
    // string: rep(i, 0, n) cout << char('a' + rnd(0, 2));
    // permutation: vi p(n); iota(all(p), 1); shuffle(all(p), rng);
}
```

```
# Stress test: runs sol and brute on random tests from gen until
# their outputs differ. Usage: bash stress.sh
for f in gen sol brute; do
    g++ -std=c++17 -O2 $f.cpp -o $f || exit 1
done
for i in $(seq 1 1000); do
    ./gen $i > in.txt
    ./sol < in.txt > sol.out || { echo "sol: crash"; exit 1; }
    ./brute < in.txt > brute.out || { echo "brute: crash"; exit 1; }
    if ! diff -w sol.out brute.out > /dev/null; then
        echo "Test $i is different. Input:"; cat in.txt
        echo "sol:"; cat sol.out
        echo "brute:"; cat brute.out
        exit 1
    fi
done
echo "1000 tests OK"
```

14.5 Interactive problems

Flush after every question. Test by typing the answers yourself, or with the testing tool next to the samples.

```
// interactive example: find a secret x in [1, n]. To "? y" the
// judge answers 1 if x >= y, else 0. Give the answer as "! x".
int ask(ll y) {
    cout << "? " << y << endl;   // endl flushes: without it the
    int reply;                   // judge never sees the question
    cin >> reply;
    return reply;
}
void solve() {   // call this from main
    ll n;
    cin >> n;
    ll lo = 1, hi = n;   // x is in [lo, hi]
    while (lo < hi) {
        ll mid = lo + (hi - lo + 1) / 2;
        if (ask(mid)) lo = mid;   // x >= mid
        else hi = mid - 1;
    }
    cout << "! " << lo << endl;
}
```

14.6 Debugging

`dbg(x)` is in the template and only prints when compiled with `-DLOCAL` (the c command). `assert(cond)` stops on the line. `-D_GLIBCXX_DEBUG` checks every `v[i]`.

Notes